

Министерство цифрового развития государственного управления  
информационных технологий и связи Республики Татарстан  
государственное автономное профессиональное образовательное учреждение  
«Международный центр компетенций –  
Казанский техникум информационных технологий и связи»

ВКР, ДП/ДР выполнена и  
защищена с оценкой

\_\_\_\_\_

Председатель ГЭК \_\_\_\_\_  
(подпись)

\_\_\_\_\_

(ФИО)

« \_\_\_\_ » \_\_\_\_\_ 20 \_\_\_\_ г

ДОПУСТИТЬ К ЗАЩИТЕ

Заместитель директора по учебной  
работе

\_\_\_\_\_ О. С. Тимофеева

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
(ДИПЛОМНАЯ РАБОТА или ДИПЛОМНЫЙ ПРОЕКТ)

Проектирование и разработка информационной системы для ресторана  
быстрого питания

Исполнитель,  
Выпускник группы

\_\_\_\_\_

\_\_\_\_\_ М.Р. Мубаракшин  
*подпись, дата*

Руководитель, *должность*

\_\_\_\_\_ Л.Г. Хрущёва  
*подпись, дата*

Казань 2026 г.

государственное автономное профессиональное образовательное учреждение  
«Международный центр компетенций –  
Казанский техникум информационных технологий и связи»

УТВЕРЖДАЮ

Заместитель директора  
по учебной работе

\_\_\_\_\_ О. С. Тимофеева

« \_\_\_\_ » \_\_\_\_\_ 20\_\_ г.

ЗАДАНИЕ  
на дипломный проект

Мубаракшину Марселю Раисовичу студенту группы 421 по специальности:  
09.02.07 Информационные системы и программирование, квалификация  
Программист

1. Тема дипломного проекта (ДП): Проектирование и разработка  
информационной системы для ресторана быстрого питания, утверждена  
приказом директора от « \_\_\_\_ » \_\_\_\_\_ 20\_\_ г. № \_\_\_\_\_

2. Срок сдачи Выпускником законченной ВКР, ДП/ДР: « \_\_\_\_ » \_\_\_\_\_ 20\_\_ г.

3. Исходные данные к ДП: нет

4. Краткое содержание или перечень подлежащих разработке вопросов:

4.1 Краткое содержание пояснительной записки к ДП:

- введение;
- аналитическая часть;
- проектирование программного продукта;
- реализация проекта;
- тестирование программного продукта;
- руководство пользователя;
- заключение;
- список использованных источников;
- приложение.

4.2 Требования к программному продукту:  
План-график выполнения ВКР, ДП/ДР:

| №<br>п/п | Наименование этапов                  | Срок выполнения<br>этапов |      | Подпись<br>руководителя,<br>консультантов |
|----------|--------------------------------------|---------------------------|------|-------------------------------------------|
|          |                                      | План                      | Факт |                                           |
| 1        | Введение                             |                           |      |                                           |
| 2        | Аналитическая часть                  |                           |      |                                           |
| 3        | Проектирование программного продукта |                           |      |                                           |
| 4        | Реализация программного продукта     |                           |      |                                           |
| 5        | Руководство пользователя             |                           |      |                                           |
| 6        | Заключение                           |                           |      |                                           |
| 7        | Сдача пояснительной записки          |                           |      |                                           |

Рассмотрено на заседании ЦК \_\_\_\_\_

Протокол № \_\_\_\_\_ от «\_\_\_» \_\_\_\_\_ 20\_\_ г.

Председатель ЦК \_\_\_\_\_  
*подпись*
*инициалы и фамилия*

Дата выдачи задания:

«\_\_\_» \_\_\_\_\_ 20\_\_ г.

Руководитель ВКР, ДП/ДР:

\_\_\_\_\_ \_\_\_\_\_ \_\_\_\_\_  
*должность*
*подпись*
*инициалы и фамилия*

Задание принял к исполнению: \_\_\_\_\_  
*подпись*
*инициалы и фамилия*

## СОДЕРЖАНИЕ

|                                                                    |    |
|--------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                     | 5  |
| 1 АНАЛИТИЧЕСКАЯ ЧАСТЬ .....                                        | 7  |
| 1.1 Описание предметной области .....                              | 7  |
| 1.2 Обзор аналогов .....                                           | 8  |
| 2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА .....                       | 13 |
| 2.1 Определение требований к системе .....                         | 13 |
| 2.2 Архитектура информационной системы .....                       | 14 |
| 2.3 Проектирование базы данных .....                               | 16 |
| 2.4 Проектирование пользовательского интерфейса .....              | 23 |
| 2.5 Диаграмма вариантов использования (Диаграмма Use – case) ..... | 25 |
| 3 РЕАЛИЗАЦИЯ ПРОЕКТА .....                                         | 29 |
| 3.1 Обоснование выбора языка и средства разработки .....           | 29 |
| 3.2 Создание проекта .....                                         | 34 |
| 4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА .....                         | 42 |
| 4.1 Основы тестирования программного продукта .....                | 42 |
| 4.2 Тест-кейс тестирование .....                                   | 43 |
| 5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ .....                                   | 48 |
| 5.1 Руководство пользователя для клиента .....                     | 48 |
| 5.2 Руководство пользователя для сотрудников кухни .....           | 56 |
| 5.3 Руководство пользователя для курьера .....                     | 60 |
| 5.4 Руководство пользователя для администратора .....              | 64 |
| ЗАКЛЮЧЕНИЕ .....                                                   | 68 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                             | 70 |
| ПРИЛОЖЕНИЕ А .....                                                 | 71 |
| ПРИЛОЖЕНИЕ Б .....                                                 | 73 |
| ПРИЛОЖЕНИЕ В .....                                                 | 74 |

## ВВЕДЕНИЕ

В современном ресторанном бизнесе автоматизация процессов обслуживания перестала быть просто дополнительным преимуществом и превратилась в необходимое условие успешного ведения бизнеса. Ежедневно по всему миру открываются новые заведения, внедряются системы онлайн-заказов и разрабатываются инновационные способы взаимодействия с клиентами. Посетители ожидают не просто качественной еды, а полноценного цифрового сервиса: возможность заказать блюда через мобильное приложение, отслеживать статус доставки в реальном времени, получать персональные предложения и участвовать в программах лояльности.

Объём инвестиций в автоматизацию ресторанной сферы продолжает уверенный рост, а бюджеты на разработку цифровых решений достигают рекордных значений. Однако далеко не каждое заведение обладает достаточной экспертизой и ресурсами, чтобы самостоятельно выстроить эффективную и надёжную цифровую инфраструктуру. Владельцы ресторанов часто обращаются к разрозненным решениям, которые не интегрируются друг с другом, что не всегда приводит к желаемому результату. Подобные попытки теряют практический смысл без единой системы, объективной аналитики и обратной связи от всех участников процесса.

Именно для ресторанов, нацеленных на получение измеримых бизнес-результатов, стремящихся к оптимизации внутренних процессов и поиску новых способов взаимодействия с гостями, и была разработана информационная система управления рестораном быстрого питания. На этой платформе администраторы, сотрудники кухни, курьеры и клиенты могут легко взаимодействовать друг с другом для эффективной работы и достижения поставленных целей.

Объект исследования – деятельность ресторана быстрого питания в части приёма заказов, управления персоналом и взаимодействия с клиентами.

Предмет исследования – процессы автоматизации заказов, управления персоналом и взаимодействия с клиентами в ресторане быстрого питания с использованием современных информационных технологий.

Цель выпускной квалификационной работы – разработка информационной системы управления рестораном быстрого питания, включающей серверное API, веб-панели для администратора и сотрудников кухни, а также мобильные приложения для клиентов и курьеров.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и исследование существующих решений для автоматизации ресторанного бизнеса.
2. Спроектировать архитектуру информационной системы, включая структуру базы данных и взаимодействие между компонентами.
3. Разработать API на платформе ASP.NET Core для обеспечения обмена данными между клиентскими приложениями и сервером.
4. Создать веб-приложение на Blazor для администратора с функциями управления сотрудниками, меню и аналитики.
5. Реализовать веб-панель для сотрудников кухни для приёма и обработки заказов.
6. Разработать мобильные приложения на Flutter для клиентов и для курьеров.
7. Провести тестирование и отладку всех компонентов системы.
8. Сформировать проектную и пользовательскую документацию.

Практическая значимость работы заключается в создании готового к внедрению программного продукта, позволяющего автоматизировать ключевые процессы ресторана быстрого питания. Разработанная система может быть использована в заведениях различного масштаба – от небольших кафе до крупных ресторанных сетей.

## 1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

### 1.1 Описание предметной области

Информационные системы для ресторанов и заведений быстрого питания играют важную роль в современном ресторанном бизнесе, предоставляя посетителям удобный доступ к меню, возможность онлайн-заказа еды, а персоналу – эффективные инструменты для управления заказами и аналитики. Такие платформы позволяют гостям заказывать еду в любое время и в любом месте, сокращая время ожидания и делая процесс посещения ресторана более комфортным. Они помогают заведениям автоматизировать приём заказов, оптимизировать работу кухни, отслеживать статистику и выстраивать долгосрочные отношения с клиентами через программы лояльности.

Современные требования к ресторанным информационным системам включают не только базовый функционал оформления заказов, но и интеграцию различных каналов взаимодействия с гостями: мобильные приложения для клиентов, веб-панели для администраторов, интерфейсы для сотрудников кухни и приложения для курьеров. Особую значимость приобретает возможность персонализации предложений на основе истории заказов и предпочтений пользователя.

Ключевыми пользователями ресторанной информационной системы могут быть:

- посетители (клиенты) – люди, желающие быстро и удобно заказать еду, а также ознакомиться с меню и акциями;
- курьеры – сотрудники службы доставки, которым необходим удобный интерфейс для получения заказов, построения маршрутов и подтверждения доставки;
- сотрудники кухни – повара и работники кухни, которые должны видеть поступившие заказы, отмечать готовность блюд и управлять процессом приготовления;

– владельцы и управляющие – лица, заинтересованные в аналитике продаж, отчётах по выручке, статистике популярных блюд и эффективности работы персонала.

В условиях высокой конкуренции на рынке общественного питания, особенно в сегменте быстрого обслуживания, наличие современной информационной системы становится не просто конкурентным преимуществом, а необходимым условием для успешного ведения бизнеса. Посетители ожидают быстрого и удобного сервиса, а владельцы – прозрачной аналитики и эффективного управления всеми процессами.

Таким образом, разработка информационной системы для кафе/ресторана быстрого питания является актуальной задачей, позволяющей объединить все аспекты деятельности заведения в едином цифровом пространстве, автоматизировать ключевые процессы и повысить качество обслуживания гостей.

## 1.2 Обзор аналогов

Анализ конкурентов позволяет выявить сильные и слабые стороны существующих решений, определить возможности для улучшения и создания уникального продукта. Рассмотрим три популярных сервиса для автоматизации ресторанов и доставки еды.

Сервис «Яндекс.Еда» (<https://eda.yandex.ru>) (Рисунок 1.1).

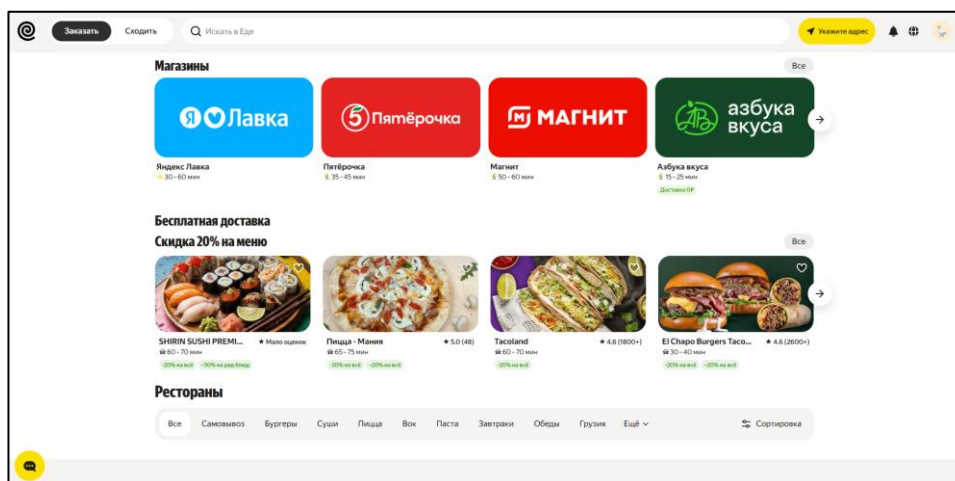


Рисунок 1.1 Система сервиса «Яндекс.Еда»



«Яндекс.Еда» – это крупнейший российский сервис доставки готовой еды из ресторанов. Платформа объединяет тысячи заведений и предлагает пользователям удобный интерфейс для заказа блюд с возможностью отслеживания статуса доставки в реальном времени.

Плюсы:

1. Огромная база ресторанов-партнёров и широкий выбор блюд.
2. Удобное мобильное приложение с интуитивным интерфейсом.
3. Быстрая доставка и возможность отслеживания курьера на карте.
4. Различные способы оплаты (карта, наличные, Apple Pay, Google Pay).
5. Интеграция с другими сервисами Яндекса (Яндекс.Карты, Яндекс.Навигатор).
6. Система промокодов и скидок для привлечения новых пользователей.

Минусы:

1. Высокая комиссия для ресторанов (до 30%).
2. Отсутствие собственной CRM-системы для ресторанов.
3. Рестораны не имеют прямого доступа к клиентской базе.
4. Проблемы с качеством доставки в пиковые часы.
5. Отсутствие персонализированных рекомендаций на основе предпочтений пользователя.

Сервис «Delivery Club» (<https://www.delivery-club.ru>) (Рисунок 1.2)

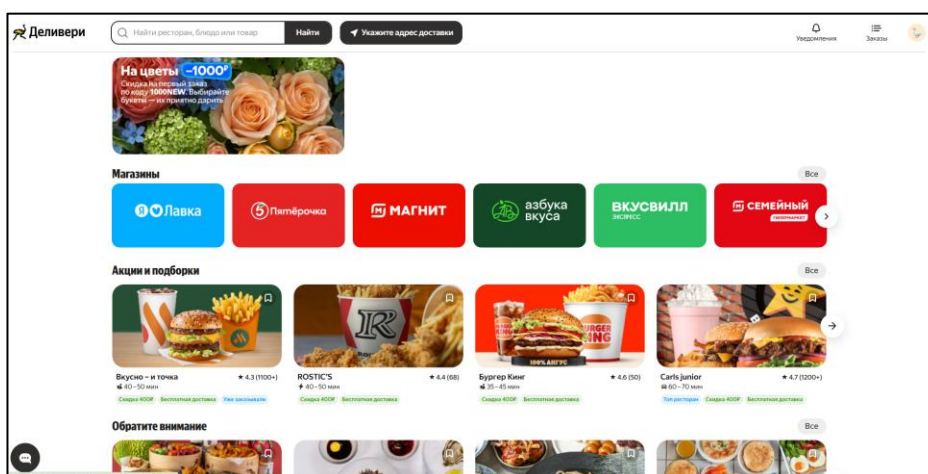


Рисунок 1.2 Система сервиса «Delivery Club»

«Delivery Club» – один из старейших и наиболее популярных сервисов доставки еды в России. Платформа предлагает заказ из ресторанов, а также доставку продуктов из магазинов.

Плюсы:

1. Широкая география покрытия (более 300 городов России).
2. Возможность заказа не только из ресторанов, но и из продуктовых магазинов.
3. Удобная система фильтрации и поиска блюд.
4. Программа лояльности «Плюс» с кешбэком.
5. Развитая служба поддержки.
6. Регулярные акции и специальные предложения.

Минусы:

1. Высокие цены на доставку в некоторых регионах.
2. Нет интеграции с внутренними системами ресторанов.
3. Длительное время обработки заказов в часы пик.
4. Рестораны не могут самостоятельно управлять меню в реальном времени.

Сервис «iiko» (<https://iiko.ru>) (Рисунок 1.3)

**Решение для вашего бизнеса от 2 500 рублей в месяц**

|                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>Доставка готовых блюд</b></p> <p>iiko помогает обеспечить четкое выполнение стандартов на всех участках бизнеса и организовать эффективную работу всех подразделений, экономия каждой секунды и не допуская ошибок.</p> |  <p><b>Столовая на предприятии или в бизнес-центре</b></p> <p>iiko поможет наладить эффективную работу столовой на предприятии или в бизнес-центре. К вашим услугам удобные инструменты для организации питания сотрудников: депозитные счета, учет денежных и продуктовых компенсаций и многое другое.</p> |  <p><b>Ресторан с быстрым обслуживанием</b></p> <p>C iiko вы сможете успешно сочетать в вашем ресторане высокую скорость обслуживания с индивидуальным подходом к каждому гостю. Система обеспечит точную и безошибочную работу персонала, а вы сможете полностью контролировать бизнес.</p> |
|                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                              |

Рисунок 1.3 Система сервиса «iiko»

«iiko» – это профессиональная система автоматизации для ресторанов, кафе и служб доставки. В отличие от агрегаторов, iiko представляет собой комплексное решение для управления бизнесом, включающее фронт-офис (работа с заказами), бэк-офис (склад, финансы) и мобильные приложения.

Плюсы:

1. Полноценная CRM-система с аналитикой и отчётами.
2. Интеграция с различными каналами заказов (агрегаторы, собственное приложение, сайт).
3. Управление складом и себестоимостью блюд.
4. Система лояльности и управления клиентской базой.
5. Мобильные приложения для официантов, курьеров и администраторов.

Минусы:

1. Высокая стоимость внедрения и обслуживания.
2. Сложность настройки и требование к квалификации персонала.
3. Ориентация преимущественно на крупные заведения.
4. Длительное внедрение и необходимость обучения сотрудников.
5. Ограниченные возможности для небольших кафе быстрого питания.
6. Устаревший интерфейс в некоторых модулях.

Для наглядности представлен сравнительный анализ рассмотренных сервисов (Таблица 1.1).

Таблица 1.1 – Сравнительный анализ аналогов

| Критерий                                      | Яндекс.Еда        | Delivery Club     | iiko    |
|-----------------------------------------------|-------------------|-------------------|---------|
| Собственное мобильное приложение для клиентов | +                 | +                 | +       |
| Приложение для курьеров                       | +                 | +                 | +       |
| Панель для кухни                              | -                 | -                 | +       |
| Административная панель                       | -                 | -                 | +       |
| CRM и аналитика                               | -                 | -                 | +       |
| Управление складом                            | -                 | -                 | +       |
| Доступная стоимость для малого бизнеса        | Низкая (комиссия) | Низкая (комиссия) | Высокая |
| Интеграция с агрегаторами                     | +                 | +                 | +       |
| Персонализация предложений                    | +                 | +                 | +       |

Таким образом, проведённый анализ показывает, что существующие на рынке решения делятся на две категории: агрегаторы доставки (Яндекс.Еда, Delivery Club), предоставляющие базовый функционал для заказа еды, и профессиональные системы автоматизации (iiko), предлагающие полный спектр инструментов для управления рестораном.

Агрегаторы удобны для пользователей, но не дают ресторанам контроля над клиентской базой и предлагают ограниченный функционал. Профессиональные системы, напротив, предоставляют все необходимые инструменты, но имеют высокую стоимость и сложность внедрения, что делает их недоступными для небольших заведений.

Разрабатываемая информационная система для кафе/ресторана быстрого питания призвана занять нишу между этими категориями, предлагая доступное по стоимости, но функциональное решение, объединяющее удобство для клиентов и полноценный контроль для владельцев. Ключевыми преимуществами должны стать: единая экосистема для всех участников процесса, интуитивно понятные интерфейсы, адаптированные под специфику быстрого обслуживания, и возможность гибкой настройки под конкретные потребности заведения.

## 2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

### 2.1 Определение требований к системе

Требования к архитектуре информационной системы.

Информационная система управления рестораном быстрого питания разработана с использованием современных технологий, что обеспечивает гибкость интеграции с внешними сервисами и масштабируемость системы. Доступ к функционалу системы предоставлен через многоуровневую систему авторизации, включая стандартную аутентификацию по логину и паролю для зарегистрированных пользователей, а также отдельные привилегированные доступы для администраторов, сотрудников кухни и курьеров.

Требования к безопасности.

Система должна соответствовать современным отраслевым стандартам безопасности и защиты персональных данных, включая шифрование конфиденциальной информации, хеширование паролей с использованием алгоритма BCrypt и регулярное резервное копирование базы данных.

Требования к отображению интерфейсов.

Разрабатываемая система включает несколько типов интерфейсов:

- веб-интерфейсы для администратора и сотрудников кухни должны корректно отображаться на стационарных компьютерах при разрешении экрана от 1024 пикселей и выше;

- мобильные приложения для клиентов и курьеров должны адаптироваться под различные размеры экранов мобильных устройств (от 320 до 414 пикселей) с учётом портретной и ландшафтной ориентации.

Функциональные требования.

Основной перечень функций системы включает:

- многоуровневую аутентификацию и авторизацию для клиентов, сотрудников и курьеров;
- регистрацию клиентов с заполнением персональных данных;
- управление меню и категориями блюд;

- формирование корзины и оформление заказов;
- управление статусами заказов;
- обработку заказов на кухне с отображением состава и отметкой готовности;
- назначение и доставку заказов курьерами с подтверждением кода доставки;
- начисление бонусных баллов и отслеживание уровня лояльности клиентов;
- формирование отчётов по выручке, популярным блюдам и загрузке залов;
- управление сотрудниками, их ролями и правами доступа.

Система обеспечивает комплексную защиту персональных данных пользователей, платёжной информации и персональных контактов, а также предотвращает несанкционированный доступ к закрытым разделам системы.

## 2.2 Архитектура информационной системы

Информационная система управления рестораном быстрого питания построена по архитектуре «клиент-сервер» с использованием микро сервисного подхода в рамках одного API-приложения. Взаимодействие между компонентами системы осуществляется через RESTful API, разработанный на платформе ASP.NET Core [1]. Все используемые технологии интегрированы между собой и обеспечивают стабильную работу без ошибок при выполнении операций.

Серверная часть системы реализована на платформе .NET 8 с использованием Entity Framework Core для взаимодействия с базой данных. В качестве системы управления базами данных (СУБД) используется PostgreSQL, обеспечивающая надёжное хранение данных и поддержку транзакций. Все данные, включая информацию о клиентах, заказах, блюдах, сохраняются в базе данных.

Клиентская часть системы представлена тремя типами приложений:

1. Веб-приложение на Blazor для администратора и сотрудников кухни. Интерфейс обеспечивает управление заказами, меню и просмотр аналитики.
2. Мобильное приложение на Flutter для клиентов, позволяющее просматривать меню, добавлять блюда в корзину, оформлять заказы.
3. Мобильное приложение на Flutter для курьеров, обеспечивающее получение заказов, отслеживание статуса доставки и подтверждение выполнения заказа с использованием кода доставки.

Все приложения взаимодействуют с сервером через единый API, что обеспечивает централизованное управление данными и согласованность бизнес-логики. API реализует аутентификацию с использованием JWT-токенов, что гарантирует безопасность передачи данных между клиентами и сервером.

Архитектурная схема системы представлена на рисунке 2.1.

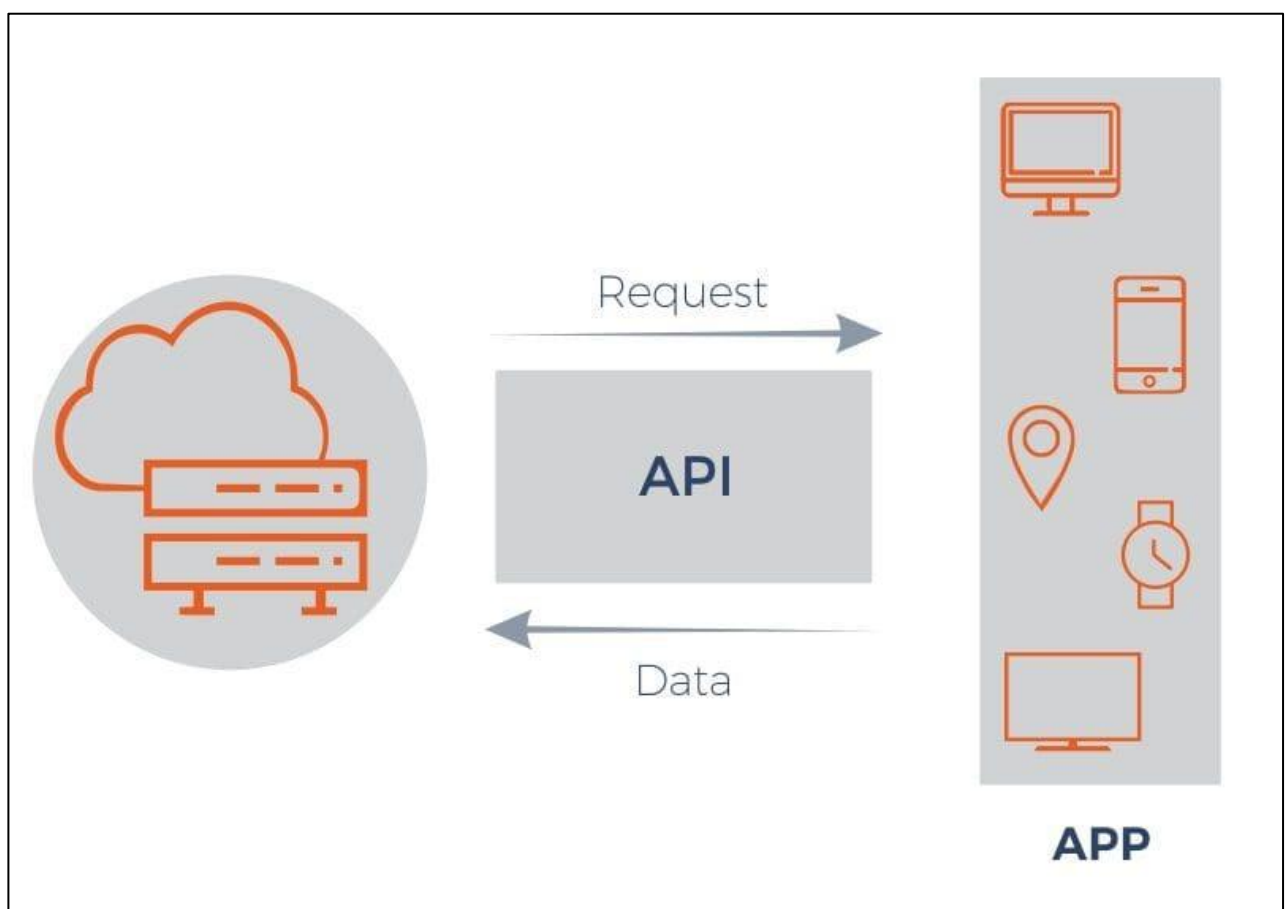


Рисунок 2.1 Архитектура информационной системы

Веб-интерфейсы и мобильные приложения полностью адаптированы для целевых устройств. При изменении размера окна браузера в веб-версии происходит плавное изменение масштаба и компоновки элементов интерфейса. Мобильные приложения корректно работают на всех современных устройствах под управлением iOS и Android. Весь функционал системы доступен на всех типах устройств в соответствии с назначением приложения.

Система оптимизирована для быстрой загрузки независимо от количества одновременных операций пользователей или нагрузки на сервер. Все пароли пользователей защищены хешированием с использованием алгоритма BCrypt [4], что делает процесс авторизации максимально безопасным. У пользователей с разными ролями (клиент, курьер, сотрудник кухни, администратор) строго разграничены права доступа, что позволяет обеспечить защиту от несанкционированного доступа.

Все элементы интерфейса функционируют согласованно и не конфликтуют между собой. Дизайн и архитектура платформы разработаны с целью обеспечения максимального удобства пользователя. Административные инструменты управления скрыты от обычных пользователей и доступны только авторизованному персоналу с соответствующими правами доступа.

### 2.3 Проектирование базы данных

В качестве системы управления базами данных (СУБД) была выбрана PostgreSQL, обеспечивающая высокую производительность, надёжность хранения данных, поддержку сложных запросов и транзакционность. Выбор данной СУБД обусловлен также бесплатностью распространения, наличием удобных инструментов администрирования (pgAdmin) и совместимостью с Entity Framework Core, используемым в серверной части приложения.

Проектирование базы данных выполнялось с использованием метода нормализации, что позволило исключить избыточность данных и обеспечить их целостность. В результате был разработан словарь данных, включающий 17



основных таблиц. В таблицах 2.1–2.17 представлено описание структуры каждой таблицы базы данных.

Таблица 2.1 – Словарь данных таблицы «dish\_categories»

| Поле          | Тип данных   | Ограничения  | Описание                |
|---------------|--------------|--------------|-------------------------|
| id            | SERIAL       | PRIMARY KEY  | Идентификатор категории |
| name          | VARCHAR(100) | NOT NULL     | Название категории      |
| display_order | INTEGER      | DEFAULT 0    | Порядок отображения     |
| active        | BOOLEAN      | DEFAULT true | Статус активности       |

Таблица 2.2 – Словарь данных таблицы «dishes»

| Поле             | Тип данных    | Ограничения   | Описание                  |
|------------------|---------------|---------------|---------------------------|
| id               | SERIAL        | PRIMARY KEY   | Идентификатор блюда       |
| name             | VARCHAR(100)  | NOT NULL      | Название блюда            |
| description      | TEXT          |               | Описание                  |
| price            | NUMERIC(10,2) | NOT NULL      | Цена                      |
| category_id      | INTEGER       | FOREIGN KEY   | Категория блюда           |
| preparation_time | INTEGER       |               | Время приготовления (мин) |
| image_url        | VARCHAR(500)  |               | Ссылка на изображение     |
| active           | BOOLEAN       | DEFAULT true  | Статус активности         |
| weight_volume    | VARCHAR(50)   |               | Вес/объем порции          |
| allergens        | TEXT          |               | Аллергены                 |
| in_stop_list     | BOOLEAN       | DEFAULT false | В стоп-листе              |

Таблица 2.3 – Словарь данных таблицы «clients»

| Поле       | Тип данных   | Ограничения  | Описание                         |
|------------|--------------|--------------|----------------------------------|
| id         | SERIAL       | PRIMARY KEY  | Уникальный идентификатор клиента |
| first_name | VARCHAR(50)  | NOT NULL     | Имя клиента                      |
| last_name  | VARCHAR(50)  | NOT NULL     | Фамилия клиента                  |
| phone      | VARCHAR(20)  | NOT NULL     | Номер телефона                   |
| email      | VARCHAR(100) | UNIQUE       | Адрес электронной почты          |
| password   | VARCHAR(100) | NOT NULL     | Хеш пароля (BCrypt)              |
| birth_date | DATE         |              | Дата рождения                    |
| created_at | TIMESTAMP    | DEFAULT      | Дата регистрации                 |
| active     | BOOLEAN      | DEFAULT true | Статус активности                |

Таблица 2.4 – Словарь данных таблицы «employees»

| Поле                 | Тип данных    | Ограничения  | Описание                 |
|----------------------|---------------|--------------|--------------------------|
| id                   | SERIAL        | PRIMARY KEY  | Уникальный идентификатор |
| first_name           | VARCHAR(50)   | NOT NULL     | Имя сотрудника           |
| last_name            | VARCHAR(50)   | NOT NULL     | Фамилия сотрудника       |
| position             | VARCHAR(50)   | NOT NULL     | Должность                |
| phone                | VARCHAR(20)   | NOT NULL     | Телефон                  |
| login                | VARCHAR(50)   | UNIQUE       | Логин для входа          |
| password             | VARCHAR(100)  | NOT NULL     | Пароль                   |
| role_id              | INTEGER       | FOREIGN KEY  | Ссылка на роль           |
| hire_date            | DATE          | NOT NULL     | Дата найма               |
| active               | BOOLEAN       | DEFAULT true | Статус активности        |
| base_salary_override | NUMERIC(12,2) | DEFAULT 0    | Индивидуальный оклад     |

|                |               |           |                       |
|----------------|---------------|-----------|-----------------------|
| bonus_override | NUMERIC(12,2) | DEFAULT 0 | Индивидуальная премия |
|----------------|---------------|-----------|-----------------------|

Таблица 2.5 – Словарь данных таблицы «employee\_roles»

| Поле        | Тип данных    | Ограничения      | Описание             |
|-------------|---------------|------------------|----------------------|
| id          | SERIAL        | PRIMARY KEY      | Идентификатор роли   |
| name        | VARCHAR(50)   | UNIQUE, NOT NULL | Название роли        |
| description | TEXT          |                  | Описание роли        |
| permissions | TEXT          |                  | Права доступа (JSON) |
| active      | BOOLEAN       | DEFAULT true     | Статус активности    |
| base_salary | NUMERIC(12,2) | DEFAULT 0        | Базовый оклад        |
| bonus       | NUMERIC(12,2) | DEFAULT 0        | Стандартная премия   |

Таблица 2.6 – Словарь данных таблицы «couriers»

| Поле         | Тип данных    | Ограничения  | Описание              |
|--------------|---------------|--------------|-----------------------|
| id           | SERIAL        | PRIMARY KEY  | Идентификатор курьера |
| first_name   | VARCHAR(50)   | NOT NULL     | Имя                   |
| last_name    | VARCHAR(50)   | NOT NULL     | Фамилия               |
| phone        | VARCHAR(20)   | NOT NULL     | Телефон               |
| vehicle_type | VARCHAR(50)   |              | Тип транспорта        |
| active       | BOOLEAN       | DEFAULT true | Статус активности     |
| base_salary  | NUMERIC(12,2) | DEFAULT 0    | Базовый оклад         |
| bonus        | NUMERIC(12,2) | DEFAULT 0    | Премия                |

Таблица 2.7 – Словарь данных таблицы «orders»

| Поле             | Тип данных    | Ограничения            | Описание             |
|------------------|---------------|------------------------|----------------------|
| id               | SERIAL        | PRIMARY KEY            | Идентификатор заказа |
| client_id        | INTEGER       | FOREIGN KEY            | Клиент               |
| order_type       | VARCHAR(20)   | DEFAULT 'в зале'       | Тип заказа           |
| status_id        | INTEGER       | FOREIGN KEY            | Статус заказа        |
| total_amount     | NUMERIC(12,2) | DEFAULT 0              | Общая сумма          |
| created_at       | TIMESTAMP     | DEFAULT NOW()          | Дата создания        |
| delivery_type    | VARCHAR(20)   | DEFAULT 'самовывоз'    | Тип доставки         |
| delivery_price   | NUMERIC(12,2) | DEFAULT 0              | Стоимость доставки   |
| delivery_code    | VARCHAR(6)    |                        | Код доставки         |
| delivery_status  | VARCHAR(50)   | DEFAULT 'не требуется' | Статус доставки      |
| delivery_address | TEXT          |                        | Адрес доставки       |
| courier_id       | INTEGER       | FOREIGN KEY            | Курьер               |
| courier_earning  | NUMERIC(12,2) | DEFAULT 0              | Заработок курьера    |

Таблица 2.8 – Словарь данных таблицы «order\_items»

| Поле           | Тип данных    | Ограничения | Описание              |
|----------------|---------------|-------------|-----------------------|
| id             | SERIAL        | PRIMARY KEY | Идентификатор позиции |
| order_id       | INTEGER       | FOREIGN KEY | Заказ                 |
| dish_id        | INTEGER       | FOREIGN KEY | Блюдо                 |
| quantity       | INTEGER       | DEFAULT 1   | Количество            |
| price_at_order | NUMERIC(10,2) | NOT NULL    | Цена на момент заказа |
| comment        | TEXT          |             | Комментарий к позиции |

Таблица 2.9 – Словарь данных таблицы «order\_status\_history»

| Поле        | Тип данных | Ограничения   | Описание                |
|-------------|------------|---------------|-------------------------|
| id          | SERIAL     | PRIMARY KEY   | Идентификатор записи    |
| order_id    | INTEGER    | FOREIGN KEY   | Заказ                   |
| status_id   | INTEGER    | FOREIGN KEY   | Статус                  |
| employee_id | INTEGER    | FOREIGN KEY   | Сотрудник (кто изменил) |
| courier_id  | INTEGER    | FOREIGN KEY   | Курьер (кто изменил)    |
| created_at  | TIMESTAMP  | DEFAULT NOW() | Время изменения         |

Таблица 2.10 – Словарь данных таблицы «order\_statuses»

| Поле        | Тип данных  | Ограничения | Описание              |
|-------------|-------------|-------------|-----------------------|
| id          | SERIAL      | PRIMARY KEY | Идентификатор статуса |
| name        | VARCHAR(50) | NOT NULL    | Название статуса      |
| description | TEXT        |             | Описание              |

Таблица 2.11 – Словарь данных таблицы «user\_levels»

| Поле               | Тип данных  | Ограничения         | Описание               |
|--------------------|-------------|---------------------|------------------------|
| id                 | SERIAL      | PRIMARY KEY         | Идентификатор записи   |
| client_id          | INTEGER     | UNIQUE, FOREIGN KEY | Клиент                 |
| current_xp         | INTEGER     | DEFAULT 0           | Текущий опыт           |
| total_xp_earned    | INTEGER     | DEFAULT 0           | Всего накоплено XP     |
| level              | INTEGER     | DEFAULT 1           | Текущий уровень        |
| level_title        | VARCHAR(50) | DEFAULT 'Новичок'   | Название уровня        |
| total_orders_count | INTEGER     | DEFAULT 0           | Количество заказов     |
| last_order_date    | TIMESTAMP   |                     | Дата последнего заказа |

Таблица 2.12 – Словарь данных таблицы «user\_xp\_history»

| Поле                | Тип данных    | Ограничения         | Описание             |
|---------------------|---------------|---------------------|----------------------|
| id                  | SERIAL        | PRIMARY KEY         | Идентификатор записи |
| client_id           | INTEGER       | FOREIGN KEY         | Клиент               |
| order_id            | INTEGER       | UNIQUE, FOREIGN KEY | Заказ                |
| xp_earned           | INTEGER       | NOT NULL            | Начислено XP         |
| order_amount        | NUMERIC(12,2) | NOT NULL            | Сумма заказа         |
| size_bonus          | NUMERIC(5,2)  | DEFAULT 0           | Бонус за размер      |
| frequency_bonus     | INTEGER       | DEFAULT 0           | Бонус за частоту     |
| calculation_formula | TEXT          |                     | Формула расчёта      |
| created_at          | TIMESTAMP     | DEFAULT NOW()       | Дата начисления      |

Таблица 2.13 – Словарь данных таблицы «xp\_activities»

| Поле        | Тип данных  | Ограничения   | Описание                 |
|-------------|-------------|---------------|--------------------------|
| id          | SERIAL      | PRIMARY KEY   | Идентификатор активности |
| client_id   | INTEGER     | FOREIGN KEY   | Клиент                   |
| xp_amount   | INTEGER     | NOT NULL      | Количество XP            |
| source_type | VARCHAR(20) | NOT NULL      | Тип источника            |
| source_id   | INTEGER     |               | Идентификатор источника  |
| earned_at   | TIMESTAMP   | DEFAULT NOW() | Время начисления         |
| expires_at  | TIMESTAMP   | NOT NULL      | Время истечения          |
| is_active   | BOOLEAN     | DEFAULT true  | Активность               |

Таблица 2.14 – Словарь данных таблицы «xp\_expirations»

| Поле       | Тип данных  | Ограничения   | Описание               |
|------------|-------------|---------------|------------------------|
| id         | SERIAL      | PRIMARY KEY   | Идентификатор списания |
| client_id  | INTEGER     | FOREIGN KEY   | Клиент                 |
| xp_amount  | INTEGER     | NOT NULL      | Списанное XP           |
| expired_at | TIMESTAMP   | DEFAULT NOW() | Время списания         |
| reason     | VARCHAR(50) | NOT NULL      | Причина списания       |

Таблица 2.15 – Словарь данных таблицы «cities»

| Поле                | Тип данных    | Ограничения  | Описание              |
|---------------------|---------------|--------------|-----------------------|
| id                  | SERIAL        | PRIMARY KEY  | Идентификатор города  |
| name                | VARCHAR(100)  | NOT NULL     | Название города       |
| base_delivery_price | NUMERIC(10,2) | DEFAULT 300  | Базовая цена доставки |
| active              | BOOLEAN       | DEFAULT true | Статус активности     |

Таблица 2.16 – Словарь данных таблицы «districts»

| Поле               | Тип данных    | Ограничения  | Описание             |
|--------------------|---------------|--------------|----------------------|
| id                 | SERIAL        | PRIMARY KEY  | Идентификатор района |
| name               | VARCHAR(100)  | NOT NULL     | Название района      |
| city_id            | INTEGER       | FOREIGN KEY  | Город                |
| delivery_surcharge | NUMERIC(10,2) | DEFAULT 300  | Надбавка за район    |
| active             | BOOLEAN       | DEFAULT true | Статус активности    |

Таблица 2.17 – Словарь данных таблицы «streets»

| Поле        | Тип данных   | Ограничения  | Описание            |
|-------------|--------------|--------------|---------------------|
| id          | SERIAL       | PRIMARY KEY  | Идентификатор улицы |
| name        | VARCHAR(200) | NOT NULL     | Название улицы      |
| district_id | INTEGER      | FOREIGN KEY  | Район               |
| active      | BOOLEAN      | DEFAULT true | Статус активности   |

Для создания структуры базы данных и заполнения её начальными данными были разработаны SQL-скрипты. Ниже представлены команды для создания основных таблиц и заполнения справочных данных.

```
CREATE TABLE clients (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100) NOT NULL,
    birth_date DATE,
    active BOOLEAN DEFAULT true);
```

-- Создание таблицы "orders"

```
CREATE TABLE orders (  
    id SERIAL PRIMARY KEY,  
    client_id INTEGER REFERENCES clients(id),  
    order_type VARCHAR(20) DEFAULT 'в_зале',  
    status_id INTEGER REFERENCES order_statuses(id) DEFAULT 1,  
    total_amount NUMERIC(12,2) DEFAULT 0,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    delivery_type VARCHAR(20) DEFAULT 'самовывоз',  
    delivery_price NUMERIC(12,2) DEFAULT 0,  
    delivery_code VARCHAR(6),  
    delivery_status VARCHAR(50) DEFAULT 'не_требуется',  
    delivery_address TEXT,  
    courier_id INTEGER REFERENCES couriers(id),  
    courier_earning NUMERIC(12,2) DEFAULT 0);
```

Заполнение таблицы "order\_statuses"

```
INSERT INTO order_statuses (id, name, description) VALUES  
(1, 'принят', 'Заказ принят в обработку'),  
(2, 'готовится', 'Заказ готовится'),  
(3, 'готов к выдаче', 'Заказ готов к выдаче'),  
(4, 'выдан клиенту (самовывоз)', 'Заказ выдан клиенту'),  
(5, 'отменен', 'Заказ отменен'),  
(6, 'готов к выдаче курьеру', 'Заказ готов к выдаче курьеру'),  
(7, 'забран курьером', 'Курьер забрал заказ'),  
(8, 'на месте', 'Курьер на месте доставки'),  
(9, 'вручен клиенту', 'Заказ вручен клиенту');
```

Полный текст SQL-скриптов для создания всех таблиц и заполнения их начальными данными приведён в Приложении А и Б.

## Примеры запросов SELECT

-- 1. Получение списка заказов клиента с детализацией (Рисунок 2.2).

SELECT

o.id, o.total\_amount,

o.created\_at,

os.name AS status,

o.delivery\_status,

o.delivery\_code

FROM orders o

INNER JOIN order\_statuses os ON o.status\_id = os.id

ORDER BY o.created\_at DESC;

| id<br>integer | total_amount<br>numeric (12,2) | created_at<br>timestamp without time zone | status<br>character varying (50) | delivery_status<br>character varying (50) | delivery_code<br>character varying (6) |
|---------------|--------------------------------|-------------------------------------------|----------------------------------|-------------------------------------------|----------------------------------------|
| 78            | 1960.00                        | 2026-03-27 10:30:57.88986                 | вручен клиенту                   | доставлен                                 | 233991                                 |
| 77            | 3740.00                        | 2026-03-26 16:12:39.619518                | готов к выдаче                   | не_требуется                              | [null]                                 |
| 76            | 2260.00                        | 2026-03-25 15:04:37.353808                | принят                           | ожидание_курьера                          | 351552                                 |

Рисунок 2.2 Список заказов клиентов с детализацией

-- 2. Получение информации об уровне клиента (Рисунок 2.3).

SELECT

c.id,

c.first\_name,

c.last\_name,

ul.current\_xp, ul.level,

ul.level\_title, ul.total\_orders\_count

FROM clients c

LEFT JOIN user\_levels ul ON ul.client\_id = c.id

WHERE c.id = 1;

| id<br>integer | first_name<br>character varying (50) | last_name<br>character varying (50) | current_xp<br>integer | level<br>integer | level_title<br>character varying (50) | total_orders_count<br>integer |
|---------------|--------------------------------------|-------------------------------------|-----------------------|------------------|---------------------------------------|-------------------------------|
| 1             | Анна                                 | Иванова                             | 0                     | 1                | Новичок                               | 0                             |

Рисунок 2.3 Получение информации об уровне клиента

Для создания структуры базы данных и заполнения её начальными данными были разработаны SQL-скрипты. Ниже представлены команды для создания основных таблиц.

Разработанная структура базы данных включает 17 таблиц, охватывающие все сущности предметной области. Использование нормализации позволило исключить избыточность данных и обеспечить их целостность. Внешние ключи гарантируют ссылочную целостность, а индексы, созданные для часто запрашиваемых полей, оптимизируют производительность запросов. Начальные данные для справочных таблиц загружены, что позволяет системе функционировать без дополнительной настройки. Спроектированная база данных обеспечивает эффективное хранение информации, быстрый доступ к данным и поддерживает масштабирование системы при росте количества пользователей и заказов.

#### 2.4 Проектирование пользовательского интерфейса

После проектирования пользовательского интерфейса были определены доступные интерфейсы и функциональные возможности для каждой категории пользователей в зависимости от их уровня доступа. Также были детализированы действия, которые каждый тип пользователя может выполнять в рамках информационной системы.

Разработанная система ролей обеспечила структурированность архитектуры и возможность гибкой адаптации функционала под специфические требования различных пользовательских групп.

В процессе анализа целевой аудитории и их потребностей были выделены четыре основные категории пользователей: администраторы ресторана, сотрудники кухни, курьеры и клиенты (зарегистрированные пользователи). Каждой из ролей назначен строго определённый набор прав доступа и функциональных ограничений.



Таблица 2.1 – Роли пользователей и доступный им функционал

| Пользователь              | Курьер                       | Администратор                | Сотрудник кухни              |
|---------------------------|------------------------------|------------------------------|------------------------------|
| Регистрация               | Авторизация                  | Авторизация                  | Авторизация                  |
| Авторизация               | Просмотр доступных заказов   | Управление сотрудниками      | Просмотр заказов на кухне    |
| Добавление блюд в корзину | Принятие заказа в доставку   | Просмотр аналитики и отчётов | Изменение статуса готовности |
| Оформление заказа         | Обновление статуса доставки  | Настройка системы            |                              |
| Просмотр истории заказов  | Подтверждение доставки кодом |                              |                              |

Администраторы обладают полным доступом ко всем разделам системы, включая управление сотрудниками и просмотр аналитики. Администраторы также имеют возможность редактировать параметры системы лояльности и управлять уровнями пользователей.

Сотрудники кухни имеют доступ к интерфейсу приёма и обработки заказов. Они могут просматривать поступившие заказы, отмечать начало приготовления и готовность блюд, а также отслеживать статус выполнения заказов.

Курьеры используют специализированное мобильное приложение, позволяющее просматривать доступные для доставки заказы, принимать их, обновлять статус доставки (забран, на месте, доставлен) и подтверждать выполнение заказа с использованием уникального кода.

Зарегистрированные клиенты имеют доступ к просмотру меню, формированию корзины, оформлению заказов, и отслеживанию статуса своих заказов. Также клиенты могут просматривать свою историю заказов и уровень лояльности.

Такое разделение прав доступа обеспечивает безопасность персональных данных пользователей и эффективное управление рабочими процессами ресторана.



## 2.5 Диаграмма вариантов использования (Диаграмма Use – case)

Диаграмма вариантов использования — это диаграмма, на которой изображаются отношения между актерами и вариантами использования [14].

Диаграмма вариантов использования (прецедентов) представляет собой граф, в вершинах которого расположены актеры или прецеденты, связи между вершинами – это разного вида отношения [14].

Актером называется любой объект, субъект или система, взаимодействующая с моделируемой системой извне [14].

Вариант использования — описание множества последовательности действий, выполняемых системой для того, чтобы актер мог получить определенный результат [15].

При разработке диаграммы вариантов использования первым делом необходимо выделить основных пользователей. В данной предметной области были выделены следующие роли:

1. Клиент – пользователь, который взаимодействует с системой через мобильное приложение. Основные действия клиента включают регистрацию и авторизацию, просмотр меню, формирование корзины, оформление заказа, отслеживание статуса заказа и просмотр истории заказов.

2. Курьер – сотрудник службы доставки, использующий специализированное мобильное приложение. Курьер может авторизоваться в системе, просматривать доступные заказы, принимать заказ в доставку, обновлять статус доставки (забран, на месте) и подтверждать доставку с использованием кода подтверждения.

3. Сотрудник кухни – работник кухни, взаимодействующий с системой через веб-интерфейс. Сотрудник кухни может авторизоваться, просматривать поступившие заказы, отмечать начало приготовления, подтверждать готовность блюд и управлять статусами заказов на кухне.

4. Администратор – пользователь с наивысшими правами доступа, работающий через веб-интерфейс. Администратор может управлять меню (добавление, редактирование, удаление блюд, управление категориями, стоп-

лист), управлять сотрудниками (добавление, редактирование, деактивация), просматривать аналитику и отчёты, управлять уровнями лояльности и настройками системы.

Рассмотрим основные варианты использования для клиента (Рисунок 2.9).

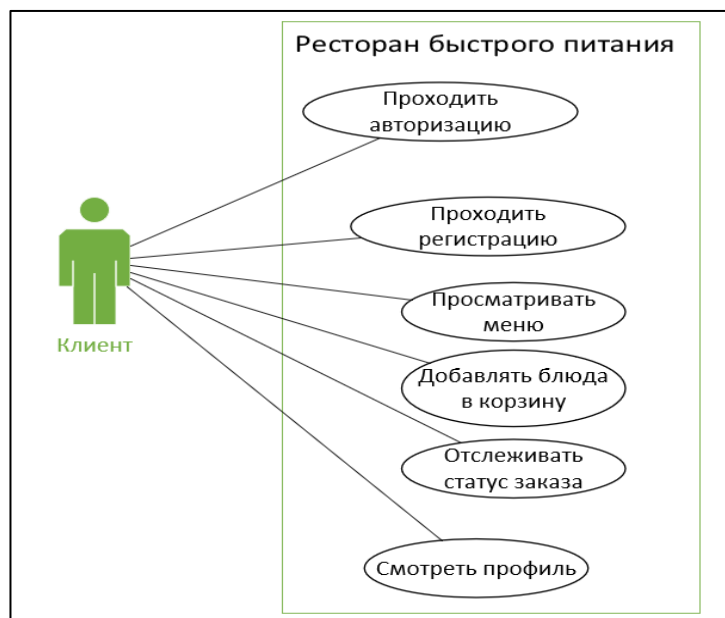


Рисунок 2.9 – Диаграмма вариантов использования для клиента

Рассмотрим также основные варианты использования для курьера (Рисунок 2.10).



Рисунок 2.10 – Диаграмма вариантов использования для курьера

Рассмотрим основные варианты использования для сотрудника кухни (Рисунок 2.11).



Рисунок 2.11 Диаграмма вариантов использования для сотрудника кухни

Рассмотрим основные варианты использования для администратора (Рисунок 2.12).

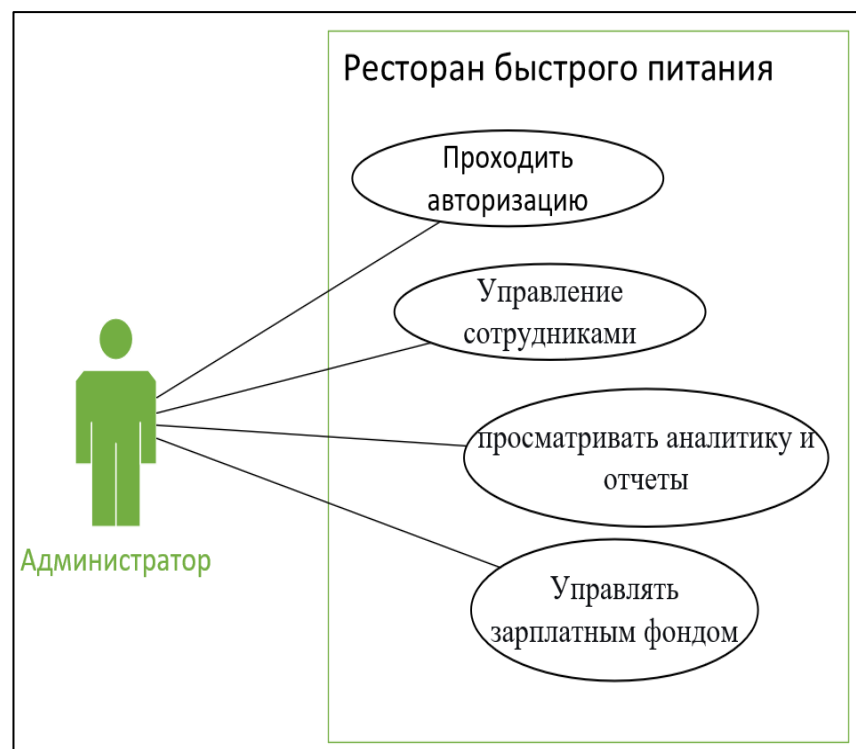


Рисунок 2.12 Диаграмма вариантов использования для администратора

На основе вышеописанных диаграмм вариантов использования создана общая диаграмма, которая объединяет в функции всех пользователей и более подробно раскрывает их (Рисунок 2.13).

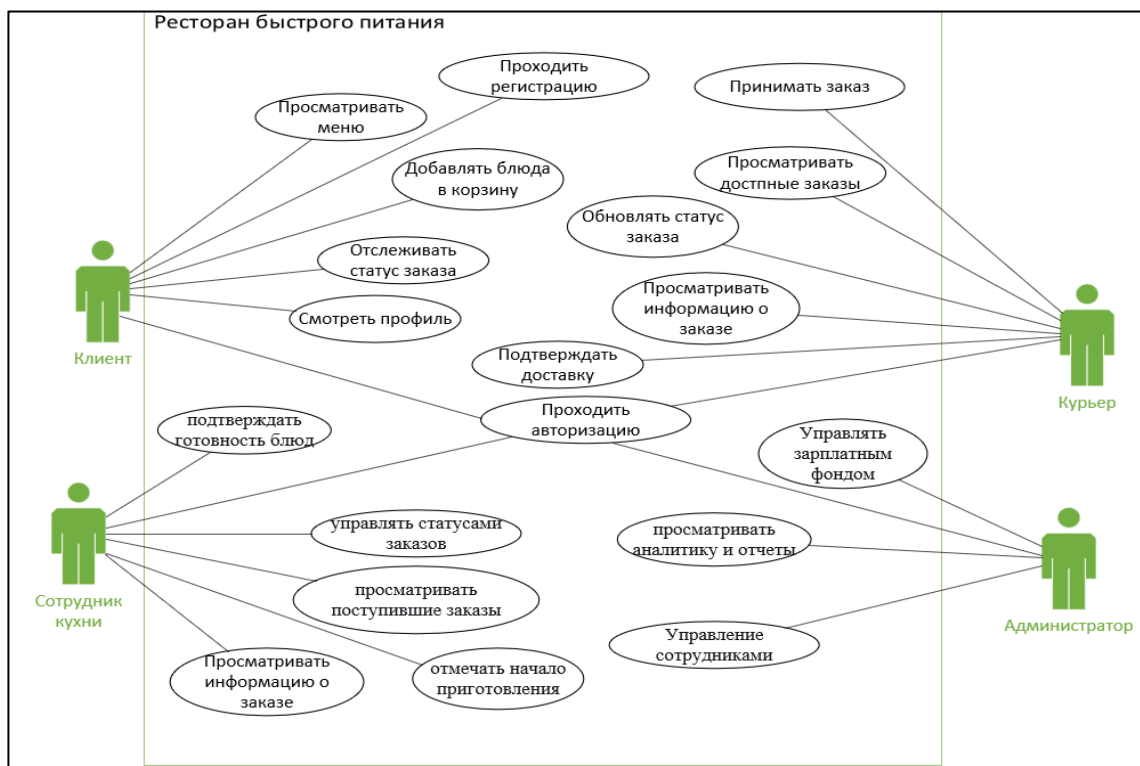


Рисунок 2.13 Общая диаграмма вариантов использования

Построенные диаграммы наглядно демонстрируют иерархию прав доступа и постепенное расширение функциональных возможностей от базового пользователя (клиента) к администратору системы. Каждая роль обладает строго определённым набором функций, что обеспечивает безопасность и эффективность работы платформы. Разграничение прав доступа позволяет защитить критически важные административные функции от несанкционированного доступа и предоставить каждому пользователю только те возможности, которые необходимы для выполнения его должностных обязанностей.

Таким образом, диаграммы вариантов использования позволяют получить полное представление о функциональных требованиях к системе, определить границы её функциональности и выявить взаимодействие между различными категориями пользователей.

## 3 РЕАЛИЗАЦИЯ ПРОЕКТА

### 3.1 Обоснование выбора языка и средства разработки

При разработке информационной системы управления рестораном быстрого питания был проведён тщательный анализ существующих технологий и инструментов, что позволило сделать обоснованный выбор языков программирования и сред разработки, оптимально соответствующих поставленным задачам. Система включает три основных компонента: серверную часть (API), веб-интерфейсы и мобильные приложения, каждый из которых предъявляет специфические требования к инструментарию разработки.

#### 3.1.1 Выбор языка для серверной части (C#)

Для реализации серверной части информационной системы был выбран язык программирования C# в связке с платформой ASP.NET Core. Данный выбор обусловлен следующими факторами:

1. C# является универсальным, безопасным в отношении типов, объектно-ориентированным языком программирования, целью которого является обеспечение продуктивности работы программистов. Для этого в нем соблюдается баланс между простотой, выразительностью и производительностью. [31]

2. Производительность и масштабируемость. ASP.NET Core является одной из самых высокопроизводительных платформ для создания веб-API. Согласно независимым тестам, приложения на ASP.NET Core демонстрируют отличные показатели пропускной способности и низкую задержку ответа, что критически важно для системы, обрабатывающей множество одновременных запросов от клиентов, курьеров и сотрудников.

3. Кроссплатформенность. ASP.NET Core работает на всех основных операционных системах (Windows, Linux, macOS), что обеспечивает гибкость при выборе серверного окружения. В рамках проекта API может быть развёрнуто как на Windows-сервере, так и на Linux-хостинге, например, на платформе Reg.ru.

4. Современный синтаксис и возможности языка. С# непрерывно развивается, предоставляя разработчикам удобные конструкции: асинхронное программирование (async/await), строгую типизацию, LINQ для удобной работы с коллекциями, запись и свойства (records, properties), а также поддержку новейших стандартов языка. Это позволяет писать надёжный, поддерживаемый и производительный код.

5. Интеграция с Entity Framework Core. Выбор С# в паре с Entity Framework Core обеспечивает удобную работу с реляционной базой данных PostgreSQL. ORM-подход позволяет взаимодействовать с базой данных через объекты языка, избегая написания большого объёма SQL-кода вручную, при этом сохраняя полный контроль над формируемыми запросами.

6. Безопасность и зрелая экосистема. Платформа .NET имеет встроенные механизмы защиты от распространённых уязвимостей (XSS, CSRF, SQL-инъекции), а также поддерживает современные методы аутентификации и авторизации, включая JWT, что было реализовано в проекте для разграничения прав доступа различных категорий пользователей.

7. Большое сообщество и поддержка. С# и .NET имеют огромное сообщество разработчиков, обширную документацию и множество готовых решений, что ускоряет процесс разработки и упрощает решение возникающих задач.

### 3.1.2 Выбор языка для мобильных приложений (Dart/Flutter)

1. Для разработки мобильных приложений (клиентского и курьерского) был выбран язык Dart с использованием фреймворка Flutter. Данный выбор обусловлен следующими преимуществами:

2. Dart – объектно-ориентированный язык программирования с сильной статической типизацией и поддержкой обобщенного программирования (шаблоны/дженерики) [32].

3. Кроссплатформенность. Flutter позволяет создавать приложения одновременно для iOS и Android из единой кодовой базы. Это существенно сокращает время и затраты на разработку, а также упрощает поддержку проекта.

В рамках дипломной работы это решение позволило разработать два полноценных мобильных приложения без необходимости изучения нативных языков (Kotlin/Java для Android и Swift/Objective-C для iOS).

4. Быстрая разработка и горячая перезагрузка. Механизм Hot Reload в Flutter позволяет мгновенно видеть изменения в интерфейсе без перекомпиляции приложения, что значительно ускоряет процесс разработки и отладки пользовательского интерфейса.

5. Высокая производительность. Flutter компилируется в нативный код (ARM для Android/iOS и JavaScript для веба), что обеспечивает высокую производительность приложений, сравнимую с нативной разработкой. Приложения работают плавно, без задержек и «зависаний».

6. Собственная система виджетов. Flutter использует собственную библиотеку виджетов, которые рендерятся с помощью Skia (графического движка), что обеспечивает единообразный внешний вид приложений на всех платформах и полный контроль над каждым пикселем интерфейса.

7. Богатая экосистема пакетов. Пакетный менеджер pub.dev предоставляет тысячи готовых решений для типовых задач: HTTP-запросы, управление состоянием (Provider), работа с базами данных, анимации, камерой, геолокацией и другими функциями, необходимыми для полноценного мобильного приложения.

8. Поддержка веба и десктопа, Flutter также позволяет компилировать приложения для веба и десктопа, что даёт возможность расширения функционала в будущем без кардинальной переработки кода.

### 3.1.3 Выбор среды разработки

Для разработки API на C# была выбрана среда Microsoft Visual Studio 2022. Данный выбор обусловлен следующими причинами:

– глубокая интеграция с .NET Visual Studio обеспечивает полную поддержку всех возможностей .NET и C#, включая встроенные шаблоны проектов, удобную отладку, профилирование производительности и

инструменты для работы с Entity Framework (миграции базы данных, визуальный конструктор моделей);

- удобная работа с Git. Встроенная поддержка систем контроля версий позволяет управлять исходным кодом непосредственно из среды разработки;

- расширенные возможности отладки. Visual Studio предоставляет мощные инструменты отладки: точки останова, просмотр состояния переменных, анализ стека вызовов, профилирование памяти и процессора, что критически важно при разработке сложной серверной логики;

- поддержка тестирования. Интеграция с тестированием и возможность запуска тестов из среды разработки упрощает процесс обеспечения качества кода;

- простота развёртывания. Visual Studio позволяет публиковать приложение непосредственно на хостинг через FTP или Azure, что было использовано при публикации API.

Visual Studio Code для мобильных приложений. Для разработки мобильных приложений на Flutter была выбрана среда Visual Studio Code. Выбор обоснован следующими факторами:

- лёгкость и быстроедействие. VS Code является лёгким редактором, который быстро запускается и не требует значительных ресурсов, что особенно важно при работе с несколькими проектами одновременно;

- мощная экосистема расширений. Существует официальное расширение Flutter, которое предоставляет полный набор инструментов: подсветку синтаксиса, автодополнение, навигацию по коду, запуск и отладку приложений, интеграцию с эмуляторами и физическими устройствами;

- интеграция с терминалом. Встроенный терминал позволяет выполнять команды Flutter (`flutter pub get`, `flutter run`, `flutter build apk`) без переключения между приложениями;

- кроссплатформенность. VS Code работает одинаково на Windows, macOS и Linux, что удобно при разработке на разных операционных системах;



- бесплатность и открытость. VS Code распространяется бесплатно, что делает его доступным для образовательных и коммерческих проектов;

- встроенная работа с Git. Удобный интерфейс для управления версиями позволяет отслеживать изменения, создавать коммиты и работать с ветками.

#### 3.1.4 Выбор системы управления базами данных

В качестве системы управления базами данных была выбрана PostgreSQL. Данный выбор обусловлен следующими преимуществами:

- соответствие требованиям проекта. PostgreSQL отлично подходит для хранения реляционных данных, которыми оперирует система: клиенты, заказы, блюда;

- бесплатность и открытость. PostgreSQL распространяется под лицензией свободного программного обеспечения, что не требует финансовых затрат на развёртывание и использование;

- надёжность и целостность данных. PostgreSQL обеспечивает поддержку транзакций, ссылочную целостность через внешние ключи, что гарантирует согласованность данных;

- поддержка JSON. Встроенная поддержка JSON-типов позволяет хранить и обрабатывать данные в гибком формате, что может быть полезно для хранения сложных структур, например, состава корзины;

- совместимость с Entity Framework Core. PostgreSQL имеет официального провайдера для Entity Framework Core (Npgsql), что обеспечивает бесшовную интеграцию с приложением на C#;

- удобный инструмент администрирования. pgAdmin предоставляет удобный графический интерфейс для управления базой данных, выполнения запросов и мониторинга.

Выбранный стек технологий (C# / ASP.NET Core / PostgreSQL / Flutter) обеспечивает высокую производительность, надёжность и масштабируемость разрабатываемой информационной системы, а также позволяет эффективно реализовать весь заявленный функционал.

## 3.2 Создание проекта

Разработка информационной системы ресторана быстрого питания осуществлялась поэтапно, начиная с создания структуры проектов, настройки окружения и заканчивая реализацией функциональных модулей. В данном разделе описывается процесс создания и начальной настройки трёх основных компонентов системы: серверного API, веб-приложения для администратора и кухни, а также мобильных приложений для клиентов и курьеров.

### 3.2.1 Создание серверного API

Серверная часть информационной системы разработана с использованием языка программирования C# на платформе ASP.NET Core 8.0 в среде разработки Microsoft Visual Studio 2022. Выбор данной технологии обусловлен высокой производительностью, кроссплатформенностью и наличием встроенных средств для создания RESTful API.

Для структурирования проекта созданы следующие папки (Рисунок 3.1):

- Controllers – содержит контроллеры, обрабатывающие HTTP-запросы;
- Models – содержит классы, описывающие структуру данных;
- Data – содержит контекст базы данных ApplicationDbContext и настройки подключения к PostgreSQL с использованием Entity Framework Core.
- Services – содержит сервисные классы, реализующие бизнес-логику;
- Migrations – автоматически генерируемые файлы миграций для управления структурой базы данных.

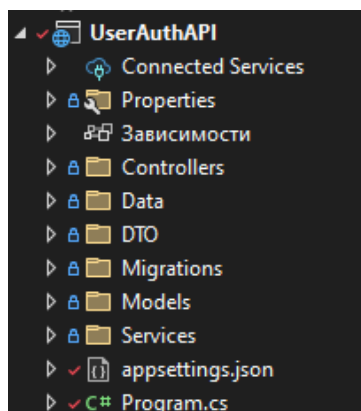


Рисунок 3.1 Структура проекта серверного API

Для работы с базой данных использовался Entity Framework Core, что позволило описывать структуру данных с помощью C# классов и выполнять миграции для синхронизации с PostgreSQL.

Пример модели данных для клиента (Рисунок 3.2).

```
namespace UserAuthAPI.Models
{
    [Table("clients")]
    Ссылка: 14
    public class Client
    {
        [Column("id")]
        Ссылка: 7
        public int Id { get; set; }

        [Column("first_name")]
        Ссылка: 6
        public string FirstName { get; set; } = string.Empty;

        [Column("last_name")]
        Ссылка: 6
        public string LastName { get; set; } = string.Empty;

        [Column("phone")]
        Ссылка: 6
        public string Phone { get; set; } = string.Empty;

        [Column("email")]
        Ссылка: 8
        public string? Email { get; set; }

        [Column("password")]
        Ссылка: 3
        public string Password { get; set; } = string.Empty;

        [Column("birth_date")]
        Ссылка: 1
        public DateTime? BirthDate { get; set; }

        [Column("created_at")]
        Ссылка: 1
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        [Column("active")]
        Ссылка: 3
        public bool Active { get; set; } = true;
    }
}
```

Рисунок 3.2 Модель клиента

Аутентификация пользователей реализована с использованием JWT-токенов. При успешном входе сервер генерирует токен, который клиент передаёт в заголовке Authorization для доступа к защищённым ресурсам (Рисунок 3.3).

```
// Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"] ?? "restaurant-api",
            ValidAudience = builder.Configuration["Jwt:Audience"] ?? "restaurant-client",
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
                builder.Configuration["Jwt:Key"] ?? "default-secret-key-min-16-chars"))
        };
    });
builder.Services.AddAuthorization();
```

Рисунок 3.3 Настройка JWT-аутентификации

Для документирования API использовался Swagger (OpenAPI). После запуска приложения документация доступна по адресу /swagger, что упрощает тестирование и интеграцию с клиентскими приложениями.

Серверное API разработано с использованием современных технологий, обеспечивающих высокую производительность и безопасность. Структура проекта организована с учётом принципов разделения ответственности, что упрощает дальнейшее сопровождение и масштабирование системы. Интеграция с PostgreSQL через Entity Framework Core позволяет эффективно управлять данными, а JWT-аутентификация обеспечивает безопасный доступ к защищённым ресурсам.

### 3.2.2 Создание веб-приложения (Blazor)

Веб-приложение для администратора и сотрудников кухни разработано с использованием фреймворка Blazor Server в среде Microsoft Visual Studio 2022. Blazor Server позволяет создавать интерактивные веб-интерфейсы на C#, что обеспечивает единообразие технологического стека с серверной частью.

Для структурирования проекта были созданы папки (Рисунок 3.4):

- Pages – содержит Razor-страницы приложения;
- Services – содержит сервисы для взаимодействия с API;
- Models – содержит модели данных.

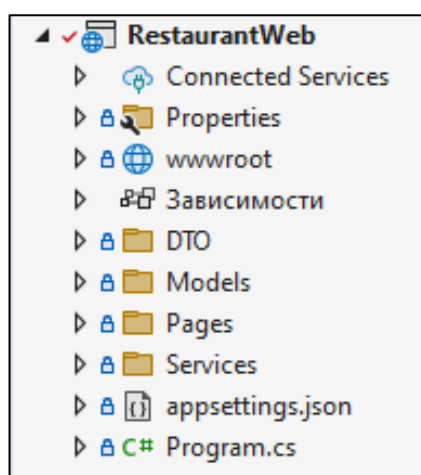
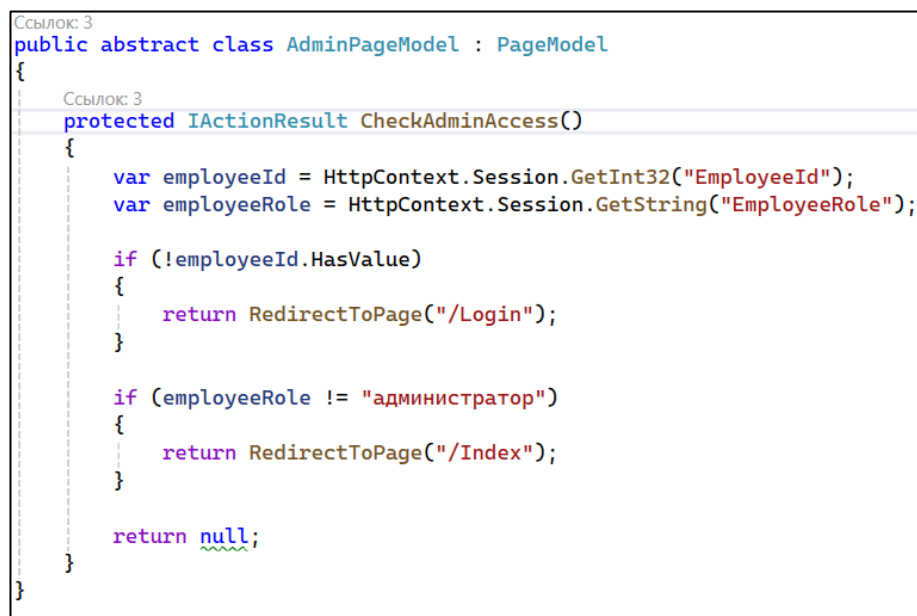


Рисунок 3.4 – Структура веб-приложения Blazor

Для взаимодействия с серверным API был создан сервис ApiService, который использует HttpClient для отправки HTTP-запросов.

Для разграничения прав доступа между администратором и сотрудниками кухни используется проверка ролей на основе данных, сохранённых в сессии (Рисунок 3.5).



```
Ссылка: 3
public abstract class AdminPageModel : PageModel
{
    Ссылка: 3
    protected IActionResult CheckAdminAccess()
    {
        var employeeId = HttpContext.Session.GetInt32("EmployeeId");
        var employeeRole = HttpContext.Session.GetString("EmployeeRole");

        if (!employeeId.HasValue)
        {
            return RedirectToPage("/Login");
        }

        if (employeeRole != "администратор")
        {
            return RedirectToPage("/Index");
        }

        return null;
    }
}
```

Рисунок 3.5 Проверка прав доступа администратора

Интерфейс веб-приложения разработан с использованием Bootstrap, что обеспечивает адаптивность и современный внешний вид. Основные страницы включают: панель администратора с карточками быстрого доступа, страницу управления сотрудниками с таблицей и модальными окнами для добавления/редактирования, страницу управления заказами для кухни с фильтрацией по статусам.

Веб-приложение на Blazor обеспечивает удобный интерфейс для администратора и сотрудников кухни. Использование единого языка C# как на сервере, так и на клиенте, упрощает разработку и поддержку кода. Разграничение прав доступа на основе ролей гарантирует безопасность административных функций, а интеграция с серверным API через HttpClient обеспечивает надёжное взаимодействие между компонентами системы.

### 3.2.3 Создание мобильных приложений (Flutter)

Мобильные приложения для клиентов и курьеров разработаны с использованием языка программирования Dart и фреймворка Flutter в среде разработки Visual Studio Code. Flutter позволяет создавать кроссплатформенные

приложения для iOS и Android из единой кодовой базы, что существенно сокращает время разработки.

Для структурирования проекта были созданы папки (Рисунок 3.6):

- lib/models – содержит классы моделей данных: Auth, Cart, Dish, Order, UserLevel, XpHistory и другие. Каждый класс содержит поля, конструкторы и методы fromJson/toJson для сериализации;

- lib/providers – содержит провайдеры для управления состоянием приложения с использованием пакета provider. Основные провайдеры: AuthProvider (аутентификация), CartProvider (корзина), MenuProvider (меню), LevelProvider (уровни и XP);

- lib/screens – содержит экраны приложения: LoginScreen, RegisterScreen, HomeScreen, MenuScreen, CartScreen, CheckoutScreen, OrdersScreen, ProfileScreen, DeliveryPaymentScreen и другие;

- lib/services – содержит сервис ApiService, отвечающий за взаимодействие с серверным API;

- lib/widgets – содержит переиспользуемые виджеты: DishCard, OrderCard, LevelProgressCard, XpHistoryCard и другие.

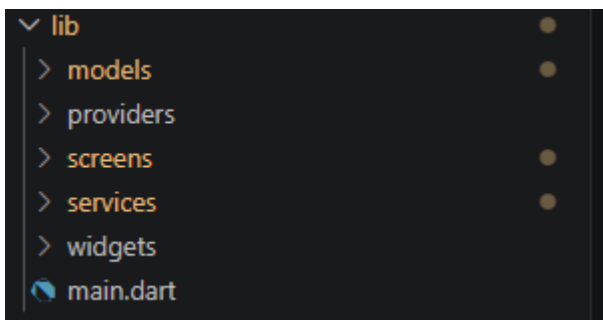


Рисунок 3.6 Структура Flutter-приложения

Для управления состоянием приложения использовался пакет provider, который позволяет легко передавать данные между виджетами и обновлять интерфейс при изменении состояния. Пример регистрации провайдеров (Рисунок 3.7).

```

return MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (context) => AuthProvider()),
    ChangeNotifierProvider(create: (context) => MenuProvider()),
    ChangeNotifierProvider(create: (context) => CartProvider()),
    ChangeNotifierProvider(create: (context) => LevelProvider()),
    ChangeNotifierProvider(
      create: (context) => ReservationProvider(apiService: apiService),

```

Рисунок 3.7 Регистрация провайдеров в приложении

Для взаимодействия с серверным API был создан класс ApiService, который инкапсулирует все HTTP-запросы. Пример метода получения уровня пользователя (Рисунок 3.8).

```

Future<UserLevel?> getUserLevel() async {
  await _ensureInitialized();

  try {
    final response = await client.get(
      Uri.parse('$baseUrl/gamification/my-level'),
      headers: _authHeaders,
    );

    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      return UserLevel.fromJson(data);
    } else {
      print('Ошибка загрузки уровня: ${response.statusCode}');
      return null;
    }
  } catch (e) {
    print('Ошибка загрузки уровня: $e');
    return null;
  }
}

```

Рисунок 3.8 Метод получения уровня пользователя

Для сборки установочного файла Android (APK) использовалась команда Flutter CLI: flutter build apk –release

Собранный APK-файл сохраняется в директории build/app/outputs/flutter-apk/ и может быть установлен на физическое устройство.

Реализация системы лояльности.

В рамках мобильного приложения для клиентов реализована система лояльности, направленная на повышение удержания пользователей и стимулирование повторных заказов. Система построена на основе накопления

опыта (XP) за совершённые заказы. Чем больше опыта набирает пользователь, тем выше становится его уровень, что даёт доступ к различным привилегиям.

Логика начисления опыта реализована в сервисе LevelService, который вызывается после успешного оформления и завершения заказа. Расчёт начисляемого опыта (XP) осуществляется по формуле, учитывающей сумму заказа и частоту покупок:

$$XP = \text{Сумма\_заказа} \times (0.5 + \text{Бонус\_за\_размер}) + \text{Бонус\_за\_частоту}$$

где:

1. 0.5 – Базовый коэффициент

2. Бонус за размер – прогрессивный коэффициент, зависящий от суммы заказа:

— от 1000 Р – +0.1;

— от 2500 Р – +0.2;

— от 5000 Р – +0.3;

— от 10000 Р – +0.5;

3. Бонус за частоту – дополнительный бонус, начисляемый за регулярность заказов:

— за заказы сегодня – +50 XP за каждый;

— при 3+ заказах за неделю – +100 XP за каждый сверх двух;

— при 5+ заказах за месяц – +200 XP за каждый сверх четырёх.

Максимальный бонус за частоту ограничен 1000 XP за один заказ.

В приложении В представлены все уровни лояльности и соответствующие им преимущества.

Примечание: Скидка не может превышать 25%, что обеспечивает экономическую целесообразность системы лояльности.

Мобильные приложения для клиентов и курьеров разработаны с использованием Flutter, что обеспечило кроссплатформенность и единообразие интерфейса на iOS и Android. Архитектура приложения построена на принципах



разделения ответственности: модели данных отделены от бизнес-логики, а управление состоянием осуществляется через провайдеры. Взаимодействие с серверным API реализовано через централизованный сервис ApiService, что упрощает поддержку и расширение функционала.

#### 3.2.4 Интеграция компонентов

Интеграция всех компонентов информационной системы (серверного API, веб-приложения и мобильных приложений) выполнена через единый RESTful API.

Интеграция компонентов информационной системы реализована через единый RESTful API, что обеспечивает централизованное управление бизнес-логикой и согласованность данных. JWT-аутентификация гарантирует безопасность обмена данными между клиентами и сервером, а разграничение прав доступа на основе ролей предотвращает несанкционированный доступ к административным функциям. Единая точка взаимодействия упрощает разработку и поддержку всех клиентских приложений.

## 4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

### 4.1 Основы тестирования программного продукта

Тестирование – один из обязательных этапов разработки программного обеспечения. Его целями выступают: проверка соответствия ПО исходным задачам, выявление и устранение ошибок, оптимизация кода. Результатом становится более эффективная программа или сайт, способный работать быстро, точно и без проблем [41].

Цель тестирования — проверка соответствия ПО предъявляемым требованиям, обеспечение уверенности в качестве ПО, поиск очевидных ошибок в программном обеспечении, которые должны быть выявлены до того, как их обнаружат пользователи программы [42].

Тестирование ПО проводится для:

- для проверки соответствия требованиям;
- для обнаружения проблем на более ранних этапах разработки и предотвращения повышения стоимости продукта;
- обнаружения вариантов использования, которые не были предусмотрены при разработке. А также взгляд на продукт со стороны пользователя;
- обеспечение стабильности и надежности работы системы;
- повышения лояльности к компании и продукту, т.к. любой обнаруженный дефект негативно влияет на доверие пользователей.

#### Вывод

Таким образом, систематическое тестирование обеспечивает соответствие разработанного приложения всем заявленным требованиям, подтверждает его способность корректно функционировать в различных режимах работы и гарантирует высокий уровень надежности конечного продукта. Результатом становится оптимизированное, стабильное программное обеспечение, готовое к эффективной эксплуатации в реальных условиях.

## 4.2 Тест-кейс тестирование

Тест-кейс — это подробная инструкция для проверки работы программы или её отдельных функций. Это как рецепт для повара или маршрут для путешественника: в тест-кейсах тестировщик описывает, что именно он проверяет, при каких условиях проводится проверка, какие действия необходимо выполнить и какой результат он рассчитывает получить [43].

В мобильном приложении была протестирована регистрация и авторизация клиента и оформлен соответствующий тест кейс (Таблица 4.1)

Таблица 4.1 – Тестирование регистрации и авторизации клиента.

|                       |                                                                                                                                                                                                                                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test Case             | 1                                                                                                                                                                                                                                                                                                                 |
| Приоритет теста       | Высокий                                                                                                                                                                                                                                                                                                           |
| Название тестирования | Регистрация нового пользователя и вход в систему                                                                                                                                                                                                                                                                  |
| Резюме испытания      | Проверка процесса регистрации нового клиента и последующей авторизации в системе                                                                                                                                                                                                                                  |
| Шаги тестирования     | 1. Открыть мобильное приложение.<br>2. Нажать кнопку «Регистрация».<br>3. Заполнить все поля формы: имя, фамилия, телефон, email, пароль.<br>4. Нажать кнопку «Зарегистрироваться».<br>5. Выполнить вход с использованием зарегистрированных учётных данных.<br>6. Проверить переход на главный экран приложения. |
| Данные тестирования   | Имя: «Иван»<br>Фамилия: «Петров»<br>Телефон: «+79001234567»<br>Email: «ivan@test.ru»<br>Пароль: «123456»                                                                                                                                                                                                          |
| Ожидаемый результат   | 1. После регистрации система автоматически выполняет вход и переводит пользователя на главный экран.<br>2. При повторном входе с корректными данными пользователь также попадает на главный экран.<br>3. В системе создаётся запись о новом пользователе.                                                         |
| Фактический результат | 1. После регистрации пользователь автоматически авторизован и перенаправлен на главный экран.<br>2. При повторном входе с корректными данными вход выполнен успешно.<br>3. В базе данных создана запись нового клиента.                                                                                           |
| Предпосылки           | 1. Открыто мобильное приложение.<br>2. Пользователь не авторизован.                                                                                                                                                                                                                                               |
| Постусловия           | Пользователь авторизован, запись о пользователе сохранена в базе данных.                                                                                                                                                                                                                                          |
| Статус (Pass/Fail)    | Pass                                                                                                                                                                                                                                                                                                              |
| Комментарии           | Проверить, что пароль сохраняется в хешированном виде (BCrypt).                                                                                                                                                                                                                                                   |

Проведённое тестирование процесса регистрации и авторизации подтвердило корректную работу модуля аутентификации. Система успешно создаёт нового пользователя, сохраняет его данные в базе данных, при этом пароль хешируется с использованием алгоритма BCrypt, что обеспечивает безопасность хранения учётных данных. Вход в систему с корректными учётными данными выполняется без ошибок, пользователь перенаправляется на главный экран приложения. При вводе некорректных данных система отображает соответствующие сообщения об ошибке, предотвращая несанкционированный доступ. Модуль аутентификации готов к эксплуатации.

В мобильном приложении была протестирована возможность формирования корзины и получения подарочного блюда при достижении суммы заказа 2000 рублей и оформлен соответствующий тест-кейс (Таблица 4.2).

Таблица 4.2 – Тестирование формирования корзины и получения подарочного блюда

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test Case             | 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Приоритет теста       | Высокий                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Название тестирования | Формирование корзины и получение подарочного блюда при достижении суммы                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Резюме испытания      | Проверка возможности добавления подарочного блюда при достижении суммы заказа 2000 рублей                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Шаги тестирования     | <ol style="list-style-type: none"> <li>1. Авторизоваться в приложении.</li> <li>2. Добавить в корзину блюда на сумму 1500 рублей.</li> <li>3. Перейти на страницу корзины.</li> <li>4. Проверить отсутствие уведомления о подарочном блюде.</li> <li>5. Добавить ещё блюда на сумму 600 рублей (общая сумма 2100 рублей).</li> <li>6. Проверить появление уведомления о возможности выбора подарочного блюда.</li> <li>7. Нажать кнопку выбора подарочного блюда.</li> <li>8. Выбрать подарочное блюдо из предложенного списка.</li> <li>9. Подтвердить выбор.</li> <li>10. Проверить отображение подарочного блюда в корзине по цене 1 рубль.</li> </ol> |
| Данные тестирования   | <p>Сумма заказа до добавления: 1500 рублей</p> <p>Сумма заказа после добавления: 2100 рублей</p> <p>Подарочное блюдо: «Картофель фри»</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Ожидаемый результат   | <ol style="list-style-type: none"> <li>1. При сумме заказа менее 2000 рублей уведомление о подарке отсутствует.</li> <li>2. При достижении суммы 2000 рублей и более появляется уведомление.</li> <li>3. После выбора подарочное блюдо добавляется в корзину по цене 1 рубль.</li> <li>4. Общая сумма заказа увеличивается на 1 рубль.</li> </ol>                                                                                                                                                                                                                                                                                                         |

Продолжение таблицы 4.2

|                       |                                                                                                                                                                                                                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Фактический результат | 1. При сумме 1500 рублей уведомление отсутствует.<br>2. При сумме 2000 рублей появилось уведомление о возможности выбора подарка.<br>3. После выбора подарочное блюдо добавлено в корзину.<br>4. Стоимость подарочного блюда составила 1 рубль, общая сумма увеличилась на 1 рубль. |
| Предпосылки           | 1. Пользователь авторизован.<br>2. В меню присутствуют подарочные блюда (категория 8).<br>3. Корзина пуста.                                                                                                                                                                         |
| Постусловия           | В корзине добавлено подарочное блюдо, сумма корзины увеличена.                                                                                                                                                                                                                      |
| Статус                | Pass                                                                                                                                                                                                                                                                                |
| Комментарии           | Проверить, что при удалении подарочного блюда возможность выбора подарка восстанавливается.                                                                                                                                                                                         |

Проведённое тестирование функционала подарочных блюд подтвердило корректную реализацию бизнес-логики. Система правильно отслеживает достижение пороговой суммы заказа и своевременно уведомляет пользователя о возможности получения подарка. Добавление подарочного блюда выполняется корректно, стоимость блюда устанавливается в 1 рубль, что соответствует требованиям маркетинговой акции. При удалении подарочного блюда система корректно обновляет состояние корзины и восстанавливает возможность повторного выбора подарка. Функционал готов к использованию.

В мобильном приложении был протестирован полный цикл доставки заказа с использованием кода подтверждения и оформлен соответствующий тест-кейс (Таблица 4.3).

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test Case             | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Приоритет теста       | Высокий                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Название тестирования | Доставка заказа с подтверждением кодом                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Резюме испытания      | Проверка полного цикла доставки заказа от принятия курьером до подтверждения получения клиентом                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Шаги тестирования     | 1. Клиент оформляет заказ с доставкой.<br>2. Система генерирует код доставки и отображает его клиенту.<br>3. Курьер авторизуется в приложении.<br>4. Курьер находит заказ в списке доступных.<br>5. Курьер нажимает кнопку «Забрать заказ».<br>6. Курьер отмечает статус «Забран курьером».<br>7. Курьер прибывает по адресу доставки и нажимает «Прибыл на место».<br>8. Курьер запрашивает у клиента код доставки.<br>9. Курьер вводит полученный код в приложение.<br>10. Система подтверждает доставку, заказ получает статус «Доставлен». |

|                       |                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Данные тестирования   | Код доставки: «123456» (генерируется автоматически)<br>Адрес доставки: ул. Бари Галеева, д. 3а                                                                                                                                                                                                                                                                                          |
| Ожидаемый результат   | 1. При оформлении заказа клиенту отображается код доставки.<br>2. Курьер видит заказ в списке доступных.<br>3. После нажатия «Забрать заказ» заказ переходит в активные доставки курьера.<br>4. После прибытия на место появляется поле для ввода кода.<br>5. При вводе корректного кода заказ получает статус «Доставлен».<br>6. При вводе неверного кода система отображает ошибку.   |
| Фактический результат | 1. Код доставки сгенерирован и отображён клиенту.<br>2. Заказ отобразился в списке доступных у курьера.<br>3. После нажатия «Забрать заказ» заказ переместился в активные доставки.<br>4. После прибытия на месте появилось поле для ввода кода.<br>5. При вводе кода «123456» заказ получил статус «Доставлен».<br>6. При вводе неверного кода система отобразила сообщение об ошибке. |
| Предпосылки           | 1. Клиент авторизован и оформил заказ с доставкой.<br>2. Курьер авторизован в приложении.<br>3. Заказ приготовлен и готов к выдаче.                                                                                                                                                                                                                                                     |
| Постусловия           | Заказ доставлен, статус изменён на «Доставлен», заказ закрыт.                                                                                                                                                                                                                                                                                                                           |
| Статус (Pass/Fail)    | Pass                                                                                                                                                                                                                                                                                                                                                                                    |
| Комментарии           | Проверить, что код доставки не может быть использован повторно после подтверждения доставки.                                                                                                                                                                                                                                                                                            |

Проведённое тестирование процесса доставки заказа с использованием кода подтверждения подтвердило корректную работу всего цикла доставки. Система успешно генерирует уникальный 6-значный код для каждого заказа, который отображается клиенту и не может быть использован повторно после успешной доставки. Курьерское приложение корректно отображает статусы заказов и позволяет последовательно обновлять их в соответствии с реальным ходом выполнения доставки. Механизм подтверждения доставки кодом обеспечивает безопасность передачи заказа и исключает возможность ошибочного или мошеннического подтверждения. Функционал доставки полностью соответствует требованиям и готов к эксплуатации.

Проведённое функциональное тестирование ключевых модулей информационной системы управления рестораном быстрого питания подтвердило корректность их работы по основным бизнес-сценариям. В ходе тестирования были проверены:

– регистрация и авторизация пользователей – система корректно создаёт учётные записи, обеспечивает безопасное хранение паролей и разграничение доступа;

– формирование корзины и получение подарочных блюд – бизнес-логика акции работает в соответствии с требованиями, пороговая сумма отслеживается корректно, подарочные блюда добавляются в корзину с установленной ценой 1 рубль;

– процесс доставки с подтверждением кодом – все этапы доставки от принятия заказа курьером до подтверждения получения клиентом выполняются без сбоев, система генерирует уникальные коды и предотвращает их повторное использование.

Все проверенные функции выполняются в соответствии с техническим заданием, система адекватно реагирует на некорректный ввод данных и обеспечивает безопасность критически важных операций (аутентификация, подтверждение доставки). Выявленные в процессе тестирования замечания были устранены, что позволило достичь стабильной работы всех модулей. Разработанная информационная система готова к внедрению в эксплуатацию.

## 5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Руководство пользователя — документ, назначение которого — предоставить людям помощь в использовании некоторой системы. Документ входит в состав технической документации на систему и, как правило, подготавливается техническим писателем [10].

Большинство руководств пользователя помимо текстовых описаний содержат изображения. В случае программного обеспечения, в руководство обычно включаются снимки экрана, при описании аппаратуры — простые и понятные рисунки. Используется стиль и язык, доступный предполагаемой аудитории, использование жаргона сокращается до минимума либо подробно объясняется [10].

### 5.1 Руководство пользователя для клиента

Для входа в систему необходимо ввести ваш email-адрес и пароль в соответствующие поля на странице авторизации (Рисунок 5.1). Убедитесь в правильности введённых данных перед нажатием кнопки «Войти». После успешной аутентификации вы попадёте на главную страницу приложения.

Рисунок 5.1 Страница авторизации

Если вы впервые пользуетесь приложением, необходимо пройти регистрацию, чтобы получить доступ ко всем возможностям платформы. В форме регистрации (Рисунок 5.2) укажите вашу электронную почту, придумайте надёжный пароль (не менее 6 символов), введите полное имя и фамилию, укажите номер телефона. После регистрации вы сможете сразу войти в систему, используя свои учётные данные.



Создайте новый аккаунт

Имя: Марсель

Фамилия: Мубаракшин

Телефон: 89674677670

Email: marselmubar@mail.ru

Пароль: .....

Подтвердите пароль: .....

Зарегистрироваться

Уже есть аккаунт? Войдите

Рисунок 5.2 Страница регистрации

После авторизации пользователь попадает на главный экран с меню ресторана (Рисунок 5.3). Меню разделено на категории для удобного поиска блюд. Вы можете просматривать описания, стоимость и изображения каждого блюда.

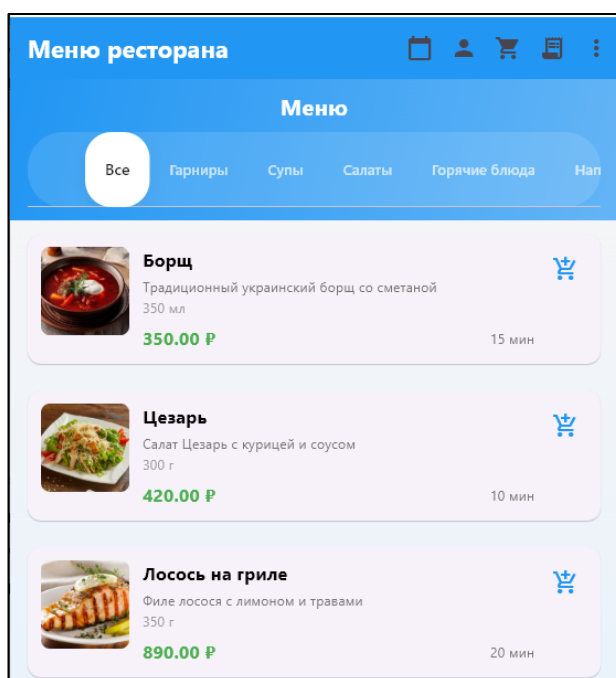


Рисунок 5.3 Меню ресторана

Чтобы добавить блюдо в корзину, нажмите на соответствующую кнопку «Добавить». После того как все необходимые блюда добавлены, перейдите на страницу корзины, нажав на иконку корзины в правом верхнем углу экрана (Рисунок 5.4).

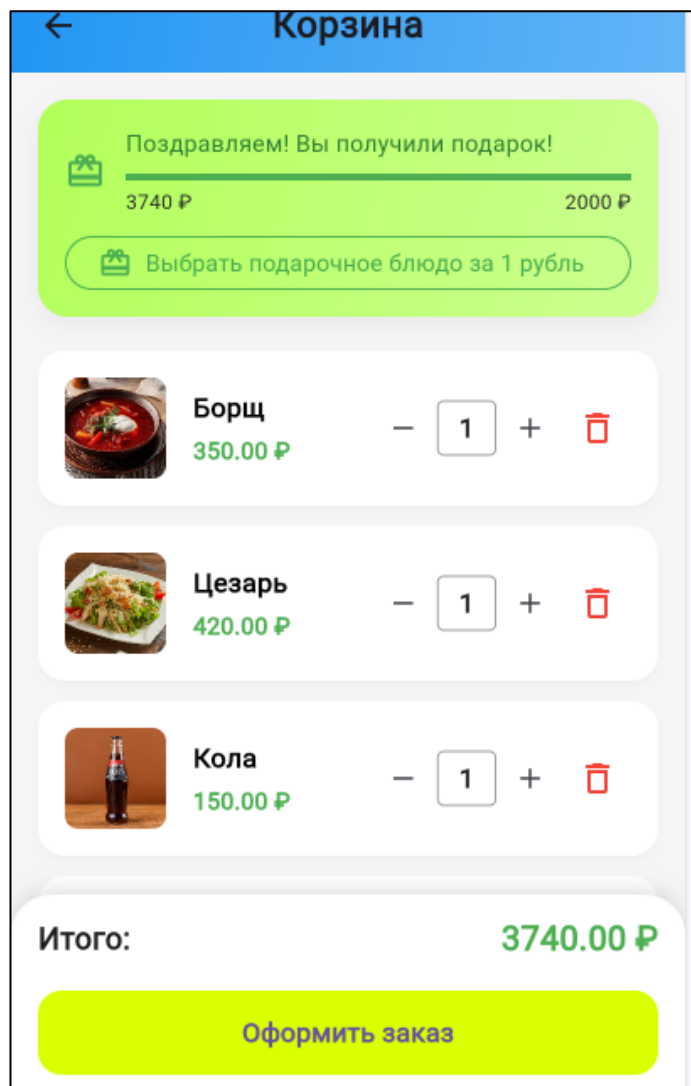


Рисунок 5.4 Страница корзины

На странице корзины отображается состав вашего заказа. Вы можете:

- изменить количество любого блюда с помощью кнопок «+» и «-»;
- удалить блюдо из корзины нажатием на кнопку с корзиной.

Особенность системы: при достижении общей суммы заказа 2000 рублей и более вам становится доступно подарочное блюдо. Система автоматически отобразит уведомление, и вы сможете выбрать одно из блюд из специальной категории «Подарочные блюда» (Рисунок 5.5). Стоимость подарочного блюда составит всего 1 рубль.

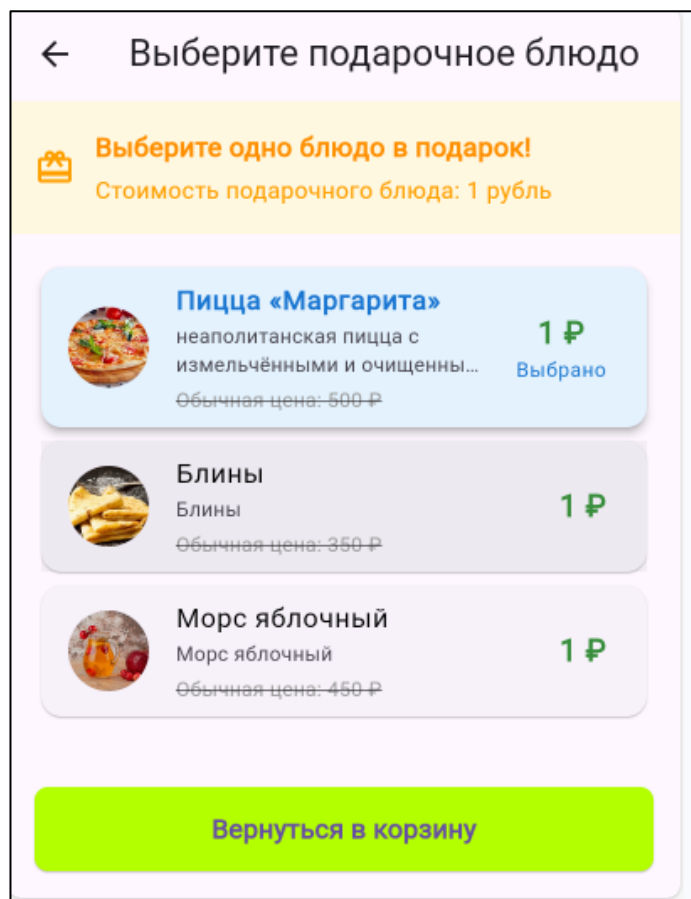


Рисунок 5.5 Страница выбора подарочного блюда

После выбора подарочного блюда оно отобразится в корзине с пометкой «Подарок» (Рисунок 5.6). Вы также можете удалить его при необходимости.

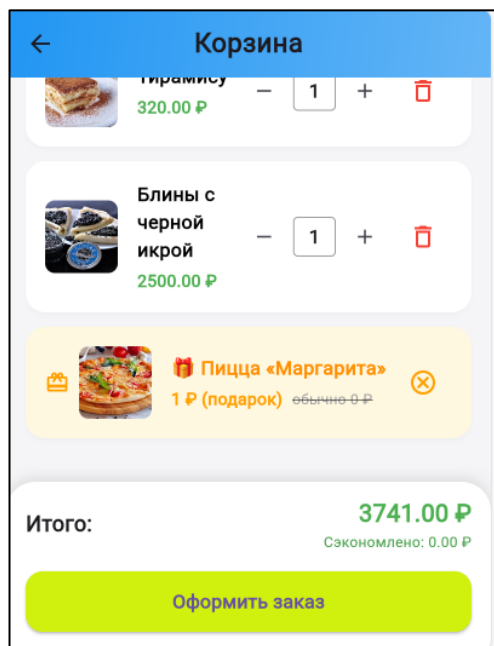


Рисунок 5.6 Страница корзины с подарочным блюдом

После того как состав корзины определён, нажмите кнопку «Оформить заказ» (Рисунок 5.7). На следующем экране вам будет предложено выбрать способ получения заказа: самовывоз или доставка курьером.

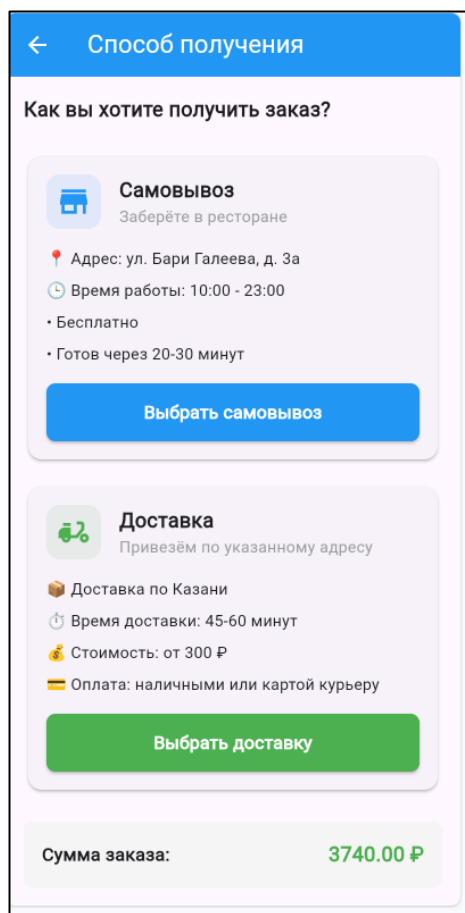


Рисунок 5.7 Страница оформления заказа

При выборе самовывоза:

1. Заполнить контактные данные (имя, телефон, email).
2. Выбрать способ оплаты (наличными или картой на кассе).
3. Подтвердите заказ.

При выборе доставки:

1. Заполните адрес доставки и, при необходимости, инструкции для курьера (Рисунок 5.8).
2. Нажать кнопку «Рассчитать доставку» — система автоматически определит стоимость доставки с учётом района и времени суток.
3. Перейти к оплате.

← Адрес доставки

Укажите адрес доставки

Мы доставляем только по Казани

Город \*

Казань

Район \*

Советский (0 Р)

Стоимость доставки в этот район: 0 Р

Улица \*

Авиации (Кульсеитово)

Выбрана улица: Авиации (Кульсеитово)

Номер дома \*

27

Квартира/Офис

12

Дополнительная информация

Подъезд

Этаж

Код домофона

оставить у двери

Рисунок 5.8 Страница доставки

На странице оплаты можно проверить:

- адрес доставки (если выбрана доставка);
- состав заказа;
- стоимость доставки;
- сумму скидки – она рассчитывается автоматически на основе вашего уровня в системе лояльности;
- итоговую сумму к оплате.

Выберите удобный способ оплаты (наличными или картой) и нажмите «Оформить заказ» (Рисунок 5.9).

**Оплата доставки**

**Информация о доставке**

Адрес: г. Казань, Советский р-н, ул. 4-й микрорайон, д. 2, кв. 1, подъезд 1, этаж 1, код домофона: 1  
Район: Советский  
Расчет доставки: База: 300 Р  
**Стоимость доставки: 300.00 Р**  
Инструкции: 1

**Выберите способ оплаты**

☒ **Наличными курьеру**  
Оплата наличными при получении заказа

☐ **Картой курьеру**  
Оплата банковской картой при получении заказа

**Итоги заказа**

|                        |                  |
|------------------------|------------------|
| Товары:                | 1660.00 Р        |
| Скидка 10%:            | - 166.00 Р       |
| Доставка:              | 300.00 Р         |
| <b>Итого к оплате:</b> | <b>1794.00 Р</b> |

Время доставки: 45-60 минут

**Оформить заказ с доставкой**

Рисунок 5.9 Страница оплаты доставки

После оформления заказа вы будете перенаправлены на страницу «Мои заказы» (Рисунок 5.10). Здесь отображается вся история ваших заказов с актуальными статусами.

**Мои заказы**

**Заказ #78** **Заказ принят**

27.3.2026 в 10:30

- Борщ x1
- Цезарь x1
- ... и ещё 1 позиций

**1960.00 Р** [Подробнее](#)

Рисунок 5.10 Страница мои заказы

Статусы заказов:

- принят – заказ получен и передан на кухню;
- готовится – заказ находится в процессе приготовления;
- готов к выдаче (для самовывоза) – заказ готов, можно забрать в ресторане;
- забран курьером (для доставки) – курьер забрал заказ из ресторана;
- на месте (для доставки) – курьер прибыл по адресу доставки;
- вручен клиенту (для доставки) – заказ доставлен и получен клиентом;
- выдан (для самовывоза) – заказ выдан клиенту.

Для заказов с доставкой на странице отображается уникальный код доставки, который необходимо назвать курьеру для подтверждения получения заказа.

Кроме того, в приложении реализована программа лояльности, которая поощряет постоянных клиентов.

Клиент может проверить свой уровень:

Для этого надо перейти в раздел «Профиль» (Рисунок 5.11). На этой странице отображается:

- ваше имя и контактные данные;
- текущий уровень и название;
- количество накопленного опыта (XP);
- прогресс до следующего уровня (визуальная шкала);
- текущая скидка и доступные бонусы (бесплатная доставка, возможность создать своё блюдо);
- история начислений опыта с детализацией расчёта.

Система начисления бонусов представлена в таблице 2.1 в разделе реализация проекта.

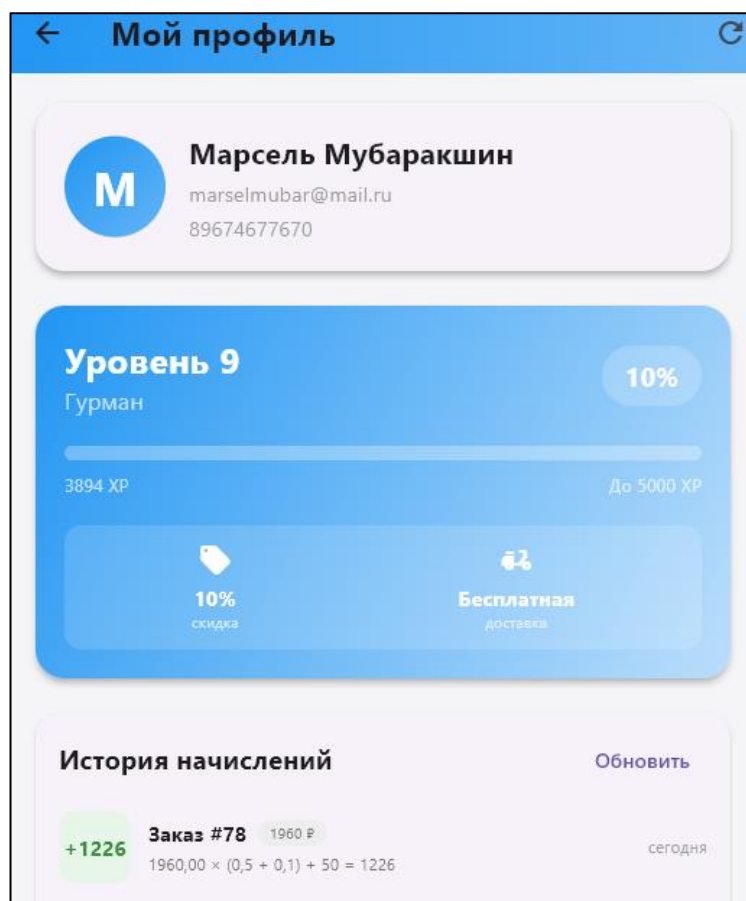


Рисунок 5.11 Страница профиля

Система лояльности разработана таким образом, чтобы поощрять как крупные заказы, так и регулярные посещения. Чем активнее вы пользуетесь приложением, тем выше ваш уровень и больше скидка.

Руководство для клиентов содержит подробное описание процесса регистрации и авторизации, работы с меню и корзиной, оформления заказов (как с самовывозом, так и с доставкой), отслеживания статуса заказов, а также принципов функционирования системы лояльности.

## 5.2 Руководство пользователя для сотрудников кухни

Для входа в систему необходимо ввести логин и пароль в соответствующие поля на странице авторизации (Рисунок 5.12). Убедитесь в правильности введённых данных перед нажатием кнопки «Войти». После успешной аутентификации вы попадёте на главную страницу веб-приложения.



**Вход в систему ресторана**

LoginInput

cook

Password

.....

Войти

Рисунок 5.12 Страница авторизации

После авторизации сотрудник кухни переходит на страницу управления заказами (Рисунок 5.13). Здесь отображаются все заказы, оформленные клиентами ресторана, в порядке их поступления.

**Управление заказами**
Обновить

Все заказы
Принятые
Готовятся
Готовы

**Заказ #78**
Принят
10:30

**Марсель Мубаракшин**  
На вынос

Борщ x1
+

Цезарь x1
+

Лосось на гриле x1
+

1 960,00 Р
Начать готовить

**Заказ #77**
Принят
16:12

**Марсель Мубаракшин**  
На вынос

Борщ x1
+

Цезарь x1
+

Кола x1
+

Тирамису x1
+

Блины с черной икрой x1
+

3 740,00 Р
Начать готовить

**Заказ #76**
Принят
15:04

**ad@mail.ru ad@mail.ru**  
На вынос

Борщ x1
+

Цезарь x1
+

Лосось на гриле x1
+

2 260,00 Р
Начать готовить

Рисунок 5.13 Страница управления заказами

На данной странице сотрудникам кухни поступают все заказы, оформленные клиентами ресторана быстрого питания.

Сотрудник кухни отслеживает и изменяет статусы заказов по мере их приготовления. Для каждого заказа доступны следующие действия в зависимости от текущего статуса (Таблица 5.1).

Таблица 5.1 – Статусы заказов

| Текущий статус | Доступное действие       | Назначение                                |
|----------------|--------------------------|-------------------------------------------|
| Принят         | «Начать готовить»        | Подтверждение начала приготовления заказа |
| Готовится      | «Заказ готов»            | Отметка о завершении приготовления        |
| Готов          | «Выдан» (для самовывоза) | Подтверждение выдачи заказа клиенту       |

Порядок работы с заказами:

1. При поступлении нового заказа – карточка заказа появляется в разделе «Принятые». Сотрудник нажимает кнопку «Начать готовить», чтобы подтвердить начало работы над заказом.

2. После приготовления заказа – сотрудник нажимает кнопку «Заказ готов». Заказ перемещается в раздел «Готовые». Для заказов с доставкой после этого статуса заказ становится доступным для курьеров.

3. После выдачи заказа (для самовывоза) – сотрудник нажимает кнопку «Выдан», заказ закрывается и удаляется из активного списка.

Фильтрация заказов:

Для удобства работы предусмотрена фильтрация заказов по статусам:

- все заказы – отображает все активные заказы;
- принятые – заказы, ожидающие начала приготовления;
- готовятся – заказы, находящиеся в процессе приготовления;
- готовы – готовые заказы, ожидающие выдачи или передачи курьеру.

Для главного сотрудника кухни доступен функционал управления меню. Для перехода в раздел управления меню необходимо нажать на соответствующую кнопку в верхней части страницы (Рисунок 5.14).

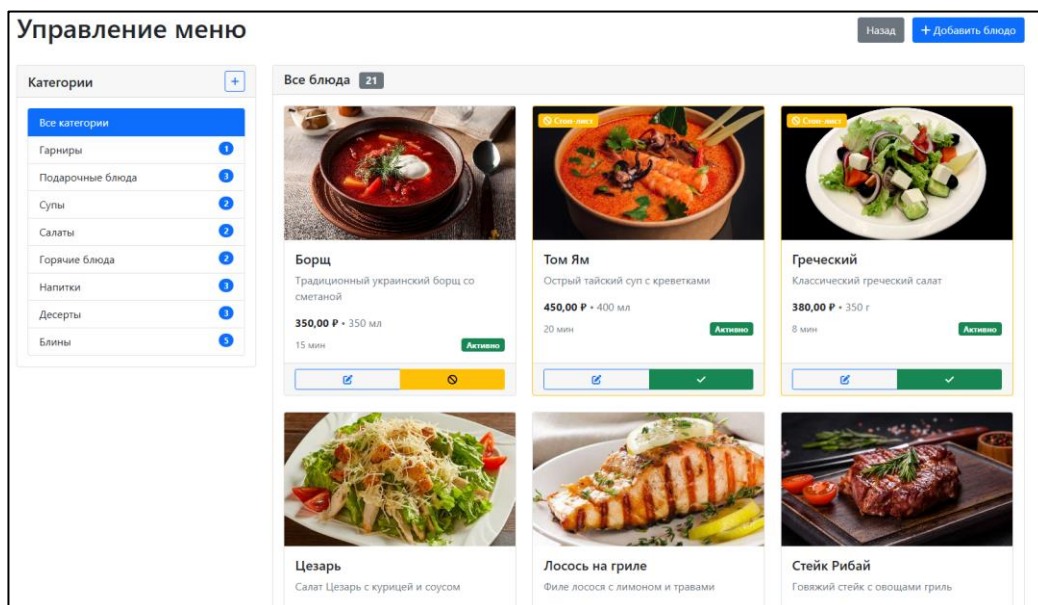


Рисунок 5.14 Страница управления меню

Функции управления меню:

1. Управление стоп-листом – при отсутствии необходимых ингредиентов сотрудник может временно скрыть блюдо из меню, чтобы клиенты не могли его заказать. Для этого необходимо нажать кнопку «В стоп-лист» рядом с соответствующим блюдом. Блюдо будет помечено специальным значком и станет недоступным для заказа. При появлении ингредиентов кнопка «Убрать из стоп-листа» возвращает блюдо в меню.

2. Редактирование блюда – при необходимости изменить цену, описание или другие параметры блюда, нажмите кнопку «Редактировать». Откроется форма для внесения изменений.

3. Добавление нового блюда – для добавления нового блюда в меню нажмите кнопку «Добавить блюдо».

При нажатии кнопки «Добавить блюдо» открывается форма добавления нового блюда (Рисунок 5.15).

**Добавить новое блюдо**

|                                                                                       |                                                                                                                      |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Название блюда *                                                                      | Описание                                                                                                             |
| <input type="text" value="Овощи гриль"/>                                              | <input type="text" value="Баклажаны, цуккини, шампиньоны, перец болгарский, томат на лаваше с маринованным луком."/> |
| Категория *                                                                           | Ссылка на изображение                                                                                                |
| <input type="text" value="Гарниры"/>                                                  | <input type="text" value="https://old-ambar.ru/upload/shop_5/7/8/1/item_781/shop_item"/>                             |
| Цена (руб) *                                                                          | Можно использовать ссылку из интернета                                                                               |
| <input type="text" value="300"/>                                                      |                                                                                                                      |
| Время приготовления (минут)                                                           | Аллергены                                                                                                            |
| <input type="text" value="5"/>                                                        | <input type="text" value="Баклажаны, цуккини, перец"/>                                                               |
| Вес/Объем                                                                             | <input type="checkbox"/> Добавить в стоп-лист                                                                        |
| <input type="text" value="400 г"/>                                                    | Блюдо в стоп-листе нельзя заказать                                                                                   |
| <input type="button" value="+ Добавить блюдо"/> <input type="button" value="Отмена"/> |                                                                                                                      |

Предпросмотр изображения



Рисунок 5.15 Страница добавления нового блюда

Сотрудник должен заполнить поля формы для добавления нового блюда (Таблица 5.2).

Таблица 5.2 – Карточка полей для добавления нового блюда

| Поле                | Описание                                                                                     | Обязательность |
|---------------------|----------------------------------------------------------------------------------------------|----------------|
| Название блюда      | Наименование, под которым блюдо будет отображаться в меню                                    | Обязательно    |
| Описание            | Краткое описание блюда, его особенности и состав                                             | Рекомендуется  |
| Категория           | Выбор категории из выпадающего списка (супы, салаты, горячие блюда, напитки, десерты и т.д.) | Обязательно    |
| Цена                | Стоимость блюда в рублях                                                                     | Обязательно    |
| Время приготовления | Ориентировочное время приготовления в минутах (отображается клиенту)                         | Рекомендуется  |
| Вес/объём           | Информация о порции (например, «250 г» или «0.5 л»)                                          | Рекомендуется  |
| Изображение         | Загрузка фотографии блюда для отображения в меню                                             | Рекомендуется  |
| Аллергены           | Информация о наличии аллергенов (молочные продукты, глютен, орехи и т.д.)                    | Рекомендуется  |

Порядок добавления блюда:

1. Заполнить все обязательные поля формы.
2. При необходимости загрузить изображение блюда.
3. Нажать кнопку «Добавить блюдо».
4. После успешного добавления блюдо появится в общем списке меню и станет доступно для заказа клиентами.

Руководство для сотрудников кухни описывает процессы приёма и обработки заказов, управления статусами, фильтрации заказов по степени готовности, а также функционал управления меню, включая добавление новых блюд и ведение стоп-листа. Документ учитывает специфику работы с различными типами заказов (самовывоз, доставка, обслуживание в зале) и содержит практические рекомендации по эффективной работе в условиях высокой нагрузки.

### 5.3 Руководство пользователя для курьера

Для входа в систему необходимо ввести логин и пароль в соответствующие поля на странице авторизации. Убедитесь в правильности введённых данных перед нажатием кнопки «Войти» (Рисунок 5.16).

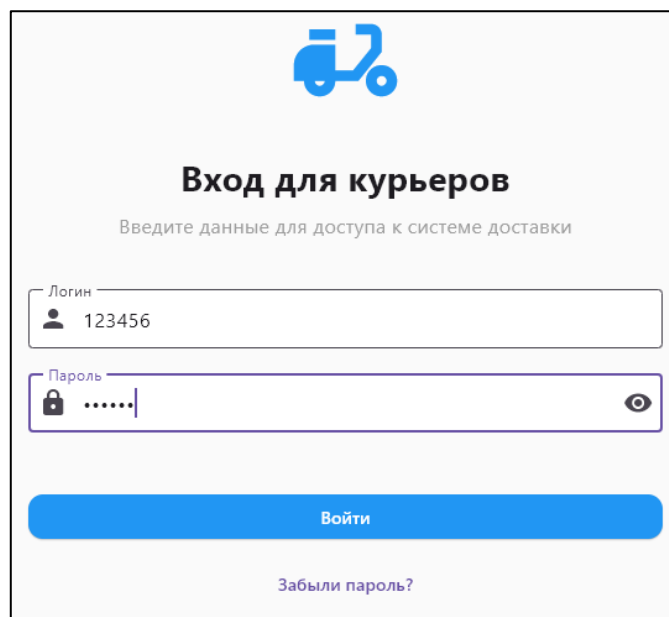


Рисунок 5.16 Страница авторизации для курьера

После успешной аутентификации вы попадёте на страницу доставки заказов, где отображаются все заказы, готовые к доставке (Рисунок 5.17). Заказы появляются в этом списке после того, как сотрудники кухни подтвердили готовность блюд.

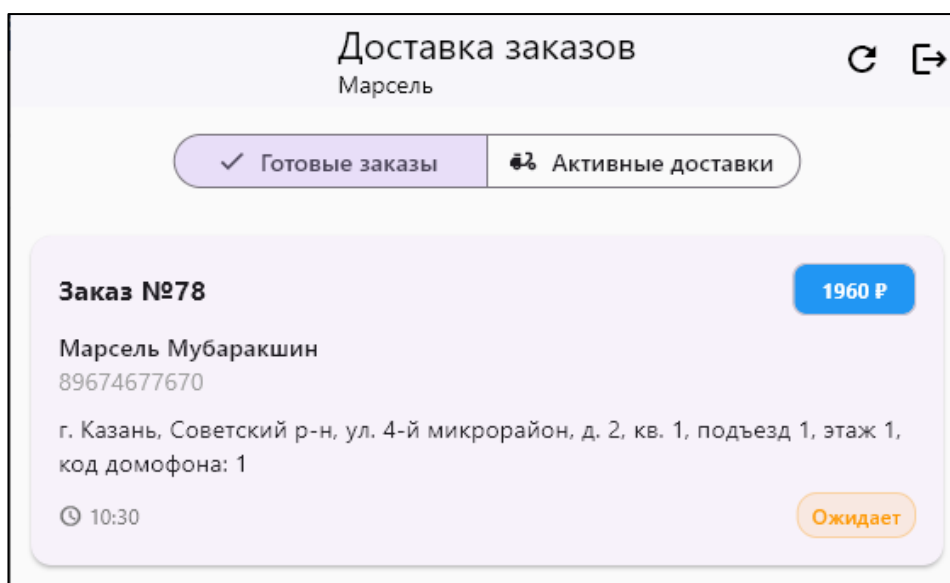


Рисунок 5.17 Доступные заказы

Курьер может просматривать все доступные заказы и выбирать те, которые удобно доставить с учётом текущего местоположения и маршрута. Для просмотра детальной информации о заказе необходимо нажать на карточку заказа.

При нажатии на карточку заказа открывается страница с подробной информацией (Рисунок 5.18).

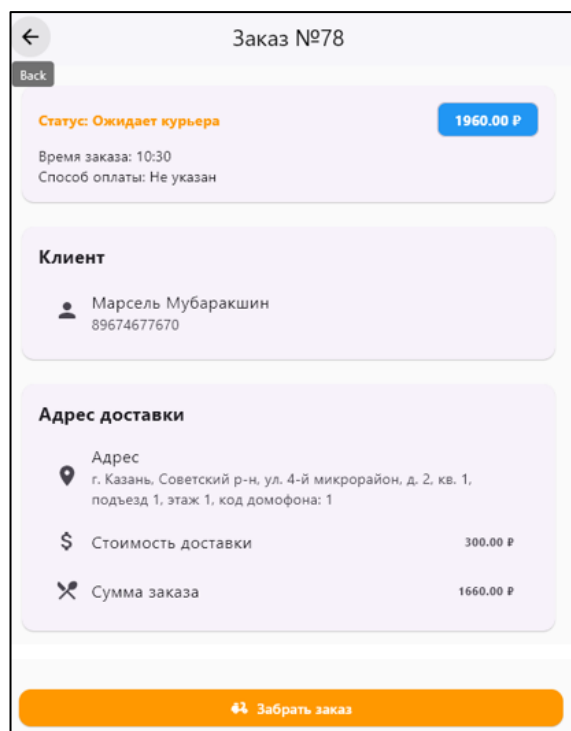


Рисунок 5.18 Страница заказа

Для принятия заказа в работу необходимо нажать кнопку «Забрать заказ». Система запросит подтверждение действия. После подтверждения заказ переходит в раздел «Активные доставки» и закрепляется за конкретным курьером.

После принятия заказа курьер переходит на страницу активных доставок (Рисунок 5.19). Здесь отображаются все заказы, которые курьер взял в работу, но ещё не доставил.

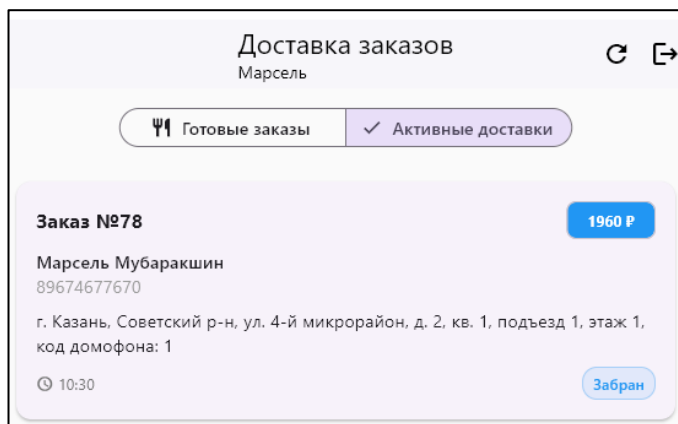


Рисунок 5.19 Страница активных доставок

Управление статусами доставки:

В процессе доставки курьер последовательно обновляет статусы заказа (Таблица 5.3).

Таблица 5.3 – Статусы заказов

| Статус          | Действие курьера                | Описание                                      |
|-----------------|---------------------------------|-----------------------------------------------|
| Забран курьером | Нажать кнопку «Забрал заказ»    | Подтверждение, что заказ получен из ресторана |
| На месте        | Нажать кнопку «Прибыл на место» | Подтверждение прибытия по адресу доставки     |
| Доставлен       | Ввести код доставки             | Подтверждение вручения заказа клиенту         |

После того как курьер прибыл на место доставки, необходимо нажать кнопку «Прибыл на место» (Рисунок 5.20). После этого откроется поле для ввода кода доставки.

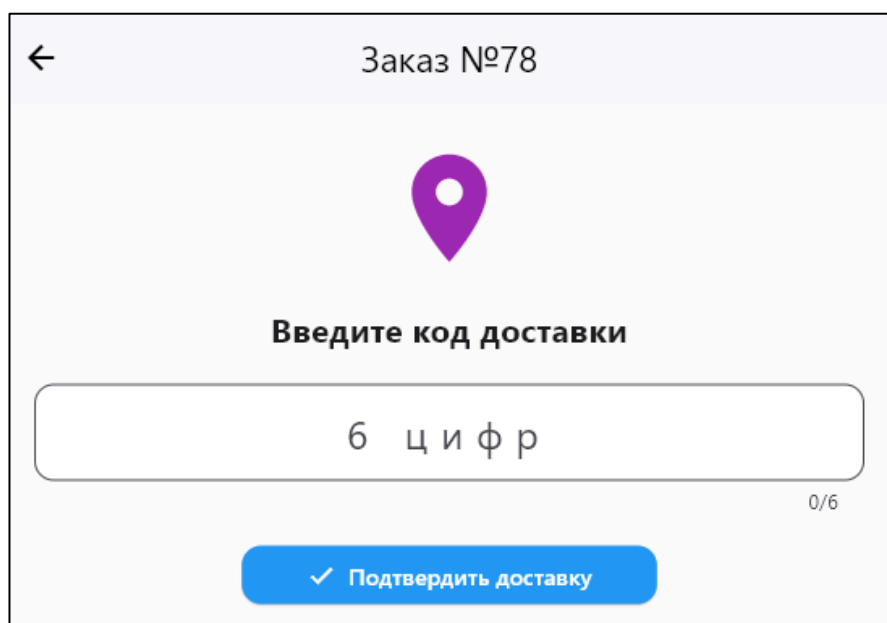


Рисунок 5.20 Подтверждение заказа доставки

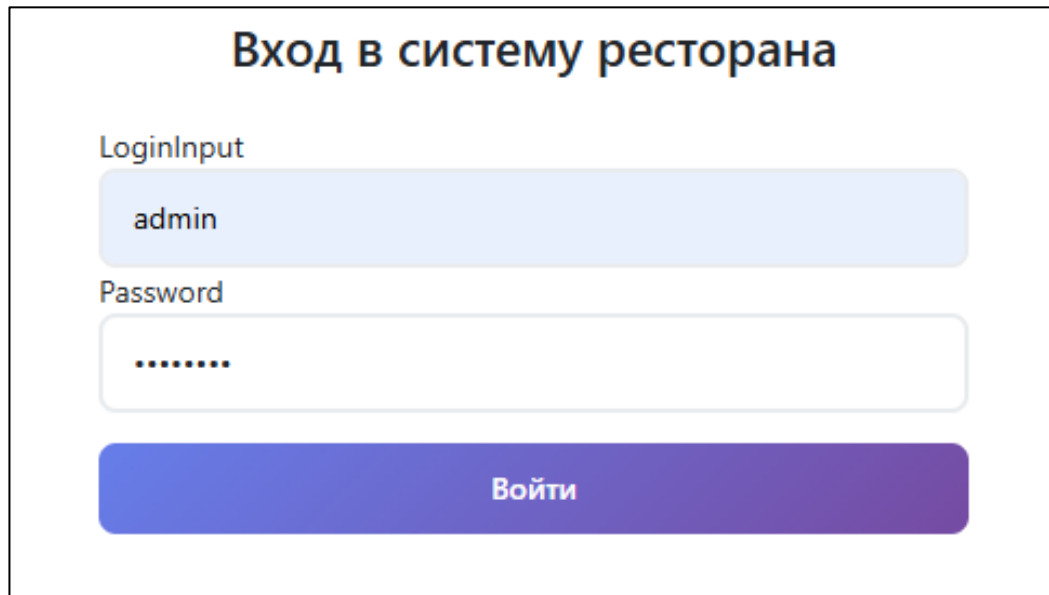
Порядок подтверждения доставки:

1. Попросите клиента назвать код доставки (код был отправлен клиенту при оформлении заказа).
2. Введите полученный 6-значный код в специальное поле.
3. Нажмите кнопку «Подтвердить доставку».

Разработанное руководство пользователя для курьеров охватывает все ключевые этапы работы с информационной системой: от авторизации в приложении до успешного завершения доставки заказа.

## 5.4 Руководство пользователя для администратора

Для входа в систему необходимо ввести логин и пароль в соответствующие поля на странице авторизации (Рисунок 5.21). Убедитесь в правильности введённых данных перед нажатием кнопки «Войти». После успешной аутентификации вы попадёте на главную страницу веб-приложения.



Вход в систему ресторана

LoginInput

admin

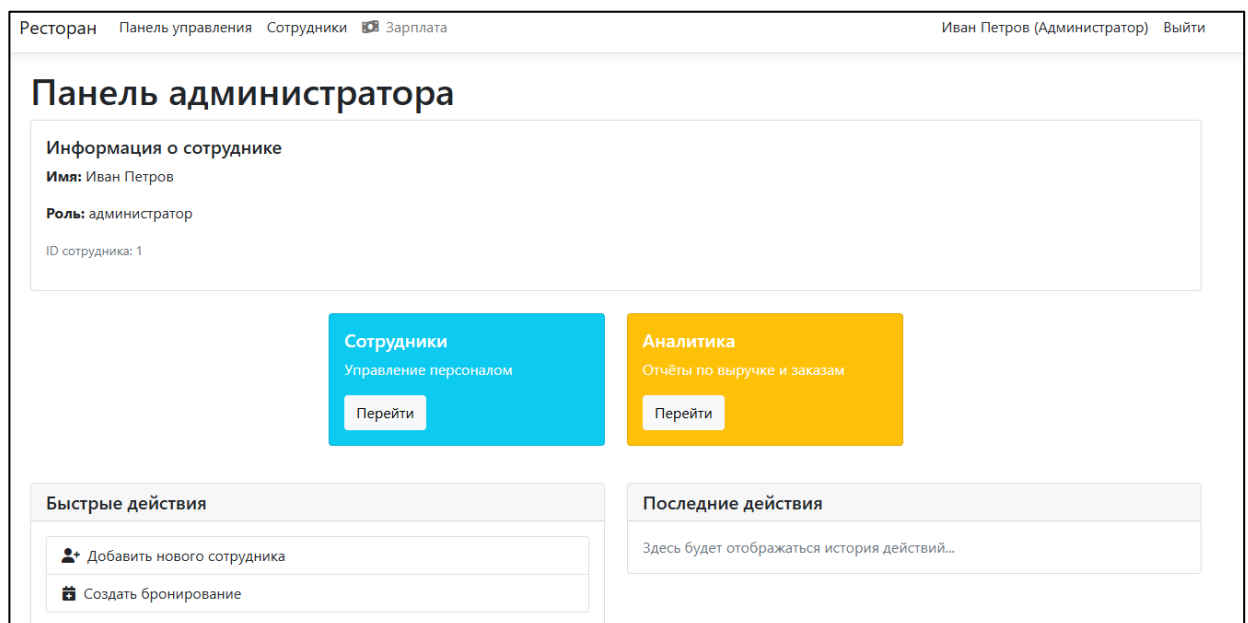
Password

.....

Войти

Рисунок 5.21 Страница авторизации

После авторизации администратор переходит на страницу панели управления (Рисунок 5.22). Здесь представлены основные разделы для управления рестораном (Таблица 5.4).



Ресторан Панель управления Сотрудники Зарплата

Иван Петров (Администратор) Выйти

### Панель администратора

**Информация о сотруднике**  
**Имя:** Иван Петров  
**Роль:** администратор  
ID сотрудника: 1

**Сотрудники**  
Управление персоналом  
Перейти

**Аналитика**  
Отчёты по выручке и заказам  
Перейти

**Быстрые действия**

- Добавить нового сотрудника
- Создать бронирование

**Последние действия**

Здесь будет отображаться история действий...

Рисунок 5.22 Панель администратора



Таблица 5.4 – Доступные разделы

| Раздел                  | Назначение                                                                                       |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Управление сотрудниками | Добавление, редактирование и деактивация учётных записей сотрудников, настройка окладов и премий |
| Аналитика и отчёты      | Просмотр финансовых показателей, динамики заказов, статистики продаж                             |
| Зарплата                | Начисление заработной платы сотрудникам, формирование отчётности                                 |

При выборе раздела «Аналитика и отчёты» администратор переходит на страницу аналитики (Рисунок 5.23).

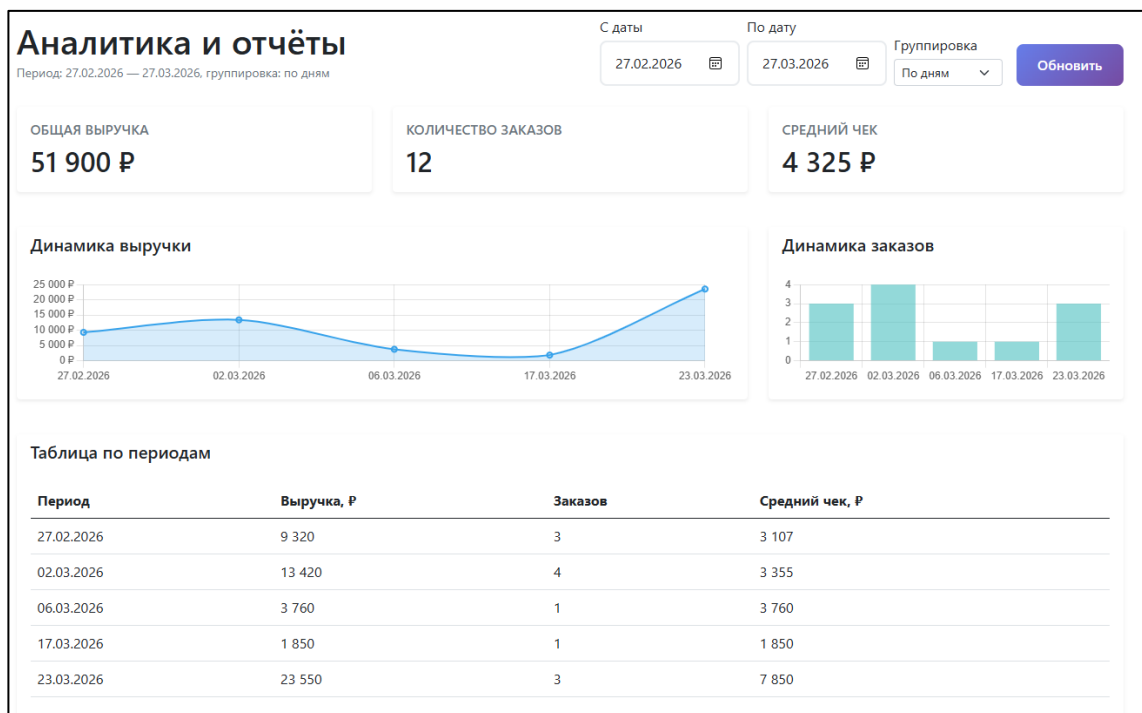


Рисунок 5.23 Страница аналитики

У администратора есть возможность фильтрации

Возможности фильтрации:

- выбор периода – возможность задать произвольный интервал дат для анализа;

- группировка данных – отображение информации по дням, месяцам или годам для выявления динамики.

Администратор может использовать данные аналитики для принятия управленческих решений: корректировки ассортимента, планирования закупок, оценки эффективности рекламных кампаний.

На странице управления сотрудниками (Рисунок 5.24) администратор может вести кадровый учёт и настраивать систему оплаты труд

| Управление сотрудниками               |                    |               |              |        |                 |            |         |                                     |
|---------------------------------------|--------------------|---------------|--------------|--------|-----------------|------------|---------|-------------------------------------|
| <a href="#">+ Добавить сотрудника</a> |                    |               |              |        |                 |            |         |                                     |
| Список сотрудников                    |                    |               |              |        |                 |            |         |                                     |
| ID                                    | ФИО                | Должность     | Телефон      | Логин  | Роль            | Дата найма | Статус  | Действия                            |
| 1                                     | Иван Петров        | Администратор | +79991112233 | admin  | Администратор   | 01.01.2024 | Активен | <a href="#">✎</a> <a href="#">✖</a> |
| 2                                     | Мария Сидорова     | Шеф-повар     | +79994445566 | cook   | Сотрудник кухни | 01.01.2024 | Активен | <a href="#">✎</a> <a href="#">✖</a> |
| 3                                     | Алексей Кузнецов   | Курьер        | +79997778899 | cook2  | Официант        | 01.02.2024 | Активен | <a href="#">✎</a> <a href="#">✖</a> |
| 4                                     | Марсель Мубаракшин | Курьер        | +79674677612 | 123456 | Курьер          | 12.12.2025 | Активен | <a href="#">✎</a> <a href="#">✖</a> |

Рисунок 5.24 Управление сотрудниками

Функции управления сотрудниками:

1. Добавление нового сотрудника:

- нажать кнопку «Добавить сотрудника»;
- заполнить форму: имя, фамилия, должность, телефон, логин, пароль, роль, дата найма;
- нажать «Сохранить». Сотрудник получит доступ к системе в соответствии с назначенной ролью.

2. Редактирование данных сотрудника:

- нажать кнопку «Редактировать» рядом с выбранным сотрудником;
- внести необходимые изменения;
- сохранить обновлённую информацию.

3. Деактивация аккаунта:

- нажать кнопку «Деактивировать» рядом с выбранным сотрудником;
- подтвердить действие. Сотрудник потеряет доступ к системе;
- при необходимости аккаунт можно активировать повторно.

В разделе «Зарплата» администратор может настраивать систему оплаты труда (Рисунок 5.25).

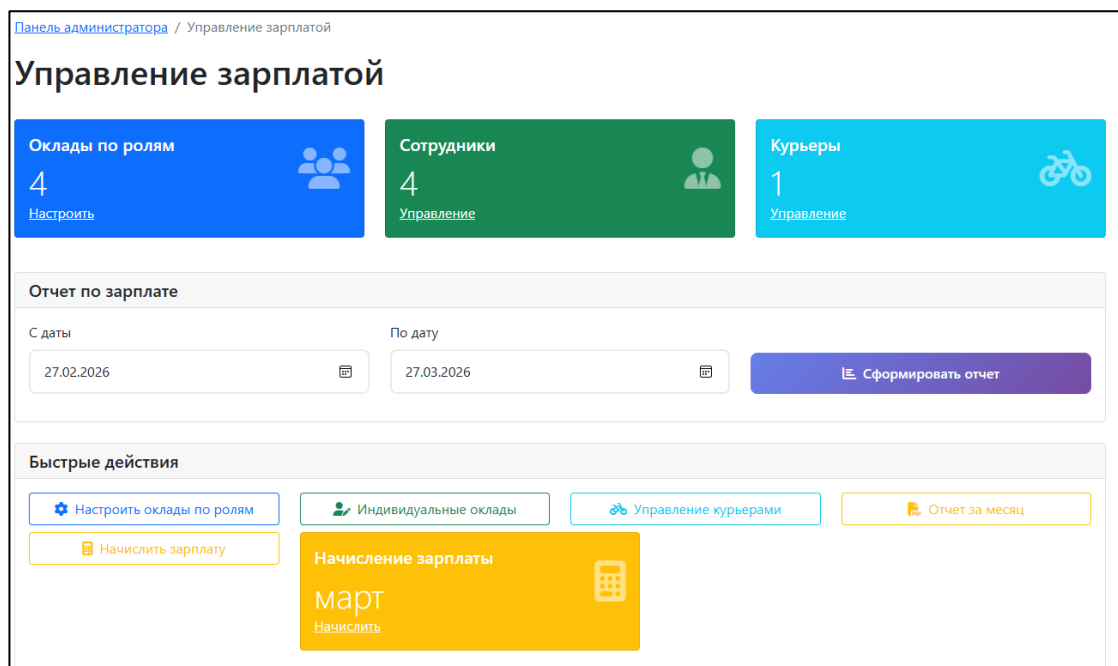


Рисунок 5.25 Страница управление зарплатой

Доступные подразделы (Таблица 5.4).

Таблица 5.4 Подразделы страницы управления зарплатой

| Подраздел             | Описание                                                                                    |
|-----------------------|---------------------------------------------------------------------------------------------|
| Оклады по ролям       | Настройка базовых окладов и премий для каждой должности                                     |
| Индивидуальные оклады | Установка персональных окладов для отдельных сотрудников (переопределение базовых значений) |
| Оклады курьеров       | Настройка оплаты для курьеров с учётом количества доставок                                  |
| Начисление зарплаты   | Формирование ведомости начисления заработной платы за выбранный период                      |
| Отчёт по зарплате     | Формирование итогового отчёта с детализацией по сотрудникам                                 |

Разработанное руководство пользователя для администратора охватывает все ключевые функции информационной системы, необходимые для эффективного управления рестораном.

Все руководства выполнены в едином стиле, содержат логическую структуру и снабжены необходимыми пояснениями. Разработанная документация позволяет пользователям с любым уровнем технической подготовки быстро освоить функционал системы и эффективно выполнять свои должностные обязанности, что способствует повышению качества обслуживания клиентов и оптимизации внутренних процессов ресторана.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта были достигнуты все поставленные задачи и цели. Разработана информационная система для ресторана быстрого питания, представляющая собой комплексное решение, объединяющее в единой экосистеме все ключевые процессы заведения: от взаимодействия с клиентами до управления персоналом и аналитики.

В процессе работы были решены следующие задачи:

- проведен анализ предметной области и существующих аналогов, что позволило выделить ключевые требования к разрабатываемой системе и определить её конкурентные преимущества;

- спроектирована архитектура информационной системы на основе RESTful API, обеспечивающая надежное и безопасное взаимодействие между всеми компонентами;

- разработана структура базы данных в PostgreSQL, обеспечивающая целостность, согласованность и высокую производительность операций с данными;

- реализован серверный API на платформе ASP.NET Core с использованием Entity Framework Core и JWT-аутентификации, выступающий центральным узлом управления бизнес-логикой;

- созданы клиентские приложения, охватывающие всех участников процесса.

- проведено тестирование ключевых модулей системы, подтвердившее их корректную работу и соответствие требованиям технического задания;

- разработана проектная и пользовательская документация, включающая руководства для всех категорий пользователей.

Реализованная система предоставляет следующие возможности:

- для клиентов – удобный инструмент для онлайн-заказа еды с возможностью самовывоза или доставки, просмотр меню и истории заказов, участие в программе лояльности;

- для курьеров – эффективную систему получения и выполнения заказов с подтверждением доставки через уникальный код;
- для сотрудников кухни – интуитивно понятный интерфейс для приёма и обработки заказов, а также управления меню;
- для администратора – полный контроль над деятельностью ресторана через панель управления, включая аналитику продаж, управление сотрудниками и настройку системы оплаты труда.

Таким образом, реализованы все основные функции, которые упростят работу персонала ресторана и поспособствуют повышению качества обслуживания клиентов. Интеграция всех участников процесса в единую платформу позволяет оптимизировать внутренние операции, снизить время обработки заказов и повысить общую эффективность бизнеса.

При разработке интерфейсов веб-приложения были использованы минималистичные цвета и элементы верстки, что позволяет сохранять концентрацию внимания пользователя и снижает утомляемость глаз при длительной работе. Мобильные приложения спроектированы с учетом адаптивности под различные размеры экранов, что обеспечивает комфортное использование на любых современных устройствах.

Все элементы интерфейса являются визуально понятными, а ключевые действия сопровождаются всплывающими подсказками. Это позволяет не тратить дополнительное время на обучение персонала работе с системой и минимизирует вероятность ошибок при выполнении повседневных операций.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ASP.NET Core в действии. Третье издание / Лок Эндрю – 2024 С 235
4. Исследование алгоритмов хеширования, используемых для аутентификации в Web-приложениях / Л.С. Адрова, П.Н. Полежаев – Оренбургский государственный университет – 198 с.
10. Руководство пользователя [Электронный ресурс] – URL: [https://ru.ruwiki.ru/wiki/Руководство\\_пользователя](https://ru.ruwiki.ru/wiki/Руководство_пользователя)
14. Визуальное моделирование систем в StarUML – КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ Рецензенты: доктор технических наук, профессор КНИТУ-КАИ Гайнутдинов В.Г. кандидат педагогических наук, доцент К(П)ФУ Халитова З.Р. – URL <https://kpfu.ru/docs/F1686173080/Kajumova.pdf>, 14 с.
15. Кватрани Т. Rational Rose 2000 и UML. Визуальное моделирование: Пер. с англ. – М.: ДМК Пресс, 2001 – 174 с. (<http://www.knigafund.ru/books/106263>)
43. Что такое тест-кейс и как его правильно оформить [Электронный ресурс] – URL: <https://skillbox.ru/media/code/chto-takoe-test-kejs-v-testirovanii-i-kak-ego-napisat/>
41. Этапы и виды тестирования [Электронный ресурс] – URL: <https://edu.sravni.ru/kursy/info/ehtapy-i-vidy-testirovaniya/>
42. Фундаментальная теория тестирования [Электронный ресурс] – URL: <https://habr.com/ru/articles/549054/?ysclid=mgmo5vlo8u74562034>
31. С# 9.0 Карманный справочник Албахари Джозеф, и Албахари, Бен Перевод с английского и редакция канд. техн. наук И.В. Красиков, 12 с
32. Основы DART Станислав Чернышев 2021, 10 с

## ПРИЛОЖЕНИЕ А

### полный код SQL-скриптов

```
-- 1. Создание таблиц
-- Таблица категорий блюд
CREATE TABLE dish_categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  display_order INTEGER DEFAULT 0,
  active BOOLEAN DEFAULT true);
-- Таблица блюд
CREATE TABLE dishes (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  price NUMERIC(10,2) NOT NULL,
  category_id INTEGER REFERENCES
dish_categories(id),
  preparation_time INTEGER,
  image_url VARCHAR(500),
  active BOOLEAN DEFAULT true,
  weight_volume VARCHAR(50),
  allergens TEXT,
  in_stop_list BOOLEAN DEFAULT false);
-- Таблица клиентов
CREATE TABLE clients (
  id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  email VARCHAR(100) UNIQUE,
  password VARCHAR(100) NOT NULL,
  birth_date DATE,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
  active BOOLEAN DEFAULT true);
-- Таблица ролей сотрудников
CREATE TABLE employee_roles (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL,
  description TEXT,
  permissions TEXT,
  active BOOLEAN DEFAULT true,
  base_salary NUMERIC(12,2) DEFAULT 0,
  bonus NUMERIC(12,2) DEFAULT 0);
-- Таблица сотрудников
CREATE TABLE employees (
  id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  position VARCHAR(50) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  login VARCHAR(50) UNIQUE,
  password VARCHAR(100) NOT NULL,
  role_id INTEGER REFERENCES
employee_roles(id),
  hire_date DATE NOT NULL,
  active BOOLEAN DEFAULT true,
  base_salary_override NUMERIC(12,2) DEFAULT
0,
  bonus_override NUMERIC(12,2) DEFAULT 0,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
-- Таблица курьеров
CREATE TABLE couriers (
  id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  vehicle_type VARCHAR(50),
  active BOOLEAN DEFAULT true,
  base_salary NUMERIC(12,2) DEFAULT 0,
  bonus NUMERIC(12,2) DEFAULT 0,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
-- Таблица статусов заказов
CREATE TABLE order_statuses (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  description TEXT);
-- Таблица заказов
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  client_id INTEGER REFERENCES clients(id),
  order_type VARCHAR(20) DEFAULT 'в_зале',
  status_id INTEGER REFERENCES
order_statuses(id) DEFAULT 1,
  total_amount NUMERIC(12,2) DEFAULT 0,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
  accepted_time TIMESTAMP,
  ready_time TIMESTAMP,
  customer_name VARCHAR(100),
  customer_phone VARCHAR(20),
  customer_email VARCHAR(100),
  comment TEXT,
  payment_method VARCHAR(50),
  delivery_type VARCHAR(20) DEFAULT
'самовывоз',
  delivery_address TEXT,
  delivery_price NUMERIC(12,2) DEFAULT 0,
  delivery_instructions TEXT,
  delivery_code VARCHAR(6),
  delivery_status VARCHAR(50) DEFAULT
'не_требуется',
  courier_id INTEGER REFERENCES couriers(id),
  assigned_at TIMESTAMP,
  picked_up_at TIMESTAMP,
  arrived_at TIMESTAMP,
  delivered_at TIMESTAMP,
  courier_earning NUMERIC(12,2) DEFAULT 0);
-- Таблица позиций заказа
CREATE TABLE order_items (
  id SERIAL PRIMARY KEY,
  order_id INTEGER REFERENCES orders(id) ON
DELETE CASCADE,
  dish_id INTEGER REFERENCES dishes(id),
  quantity INTEGER DEFAULT 1,
  price_at_order NUMERIC(10,2) NOT NULL,
```

```

comment TEXT);
-- Таблица истории статусов заказов
CREATE TABLE order_status_history (
  id SERIAL PRIMARY KEY,
  order_id INTEGER REFERENCES orders(id) ON
DELETE CASCADE,
  status_id INTEGER REFERENCES
order_statuses(id),
  employee_id INTEGER REFERENCES
employees(id),
  courier_id INTEGER REFERENCES couriers(id),
  notes TEXT,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
-- Таблица уровней пользователей (геймификация)
CREATE TABLE user_levels (
  id SERIAL PRIMARY KEY,
  client_id INTEGER UNIQUE REFERENCES
clients(id) ON DELETE CASCADE,
  current_xp INTEGER NOT NULL DEFAULT 0,
  total_xp_earned INTEGER NOT NULL DEFAULT
0,
  level INTEGER NOT NULL DEFAULT 1,
  level_title VARCHAR(50) NOT NULL DEFAULT
'Новичок',
  total_orders_count INTEGER NOT NULL
DEFAULT 0,
  last_order_date TIMESTAMP,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
-- Таблица истории начисления опыта
CREATE TABLE user_xp_history (
  id SERIAL PRIMARY KEY,
  client_id INTEGER REFERENCES clients(id) ON
DELETE CASCADE,
  order_id INTEGER UNIQUE REFERENCES
orders(id) ON DELETE CASCADE,
  xp_earned INTEGER NOT NULL,
  order_amount NUMERIC(12,2) NOT NULL,
  size_bonus NUMERIC(5,2) DEFAULT 0,
  frequency_bonus INTEGER DEFAULT 0,
  calculation_formula TEXT,
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
-- Таблица активностей XP (для учета срока
действия)
CREATE TABLE xp_activities (
  id SERIAL PRIMARY KEY,
  client_id INTEGER REFERENCES clients(id),
  xp_amount INTEGER NOT NULL,

```

```

  source_type VARCHAR(20) NOT NULL
DEFAULT 'order',
  source_id INTEGER,
  earned_at TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  expires_at TIMESTAMP NOT NULL,
  is_active BOOLEAN DEFAULT true
);
-- Таблица списаний XP
CREATE TABLE xp_expirations (
  id SERIAL PRIMARY KEY,
  client_id INTEGER REFERENCES clients(id),
  xp_amount INTEGER NOT NULL,
  expired_at TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  reason VARCHAR(50) NOT NULL DEFAULT
'timeout'
);
-- Таблица городов для доставки
CREATE TABLE cities (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  base_delivery_price NUMERIC(10,2) DEFAULT
300,
  active BOOLEAN DEFAULT true
);
-- Таблица районов
CREATE TABLE districts (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  city_id INTEGER REFERENCES cities(id),
  delivery_surcharge NUMERIC(10,2) DEFAULT
300,
  active BOOLEAN DEFAULT true
);
-- Таблица улиц
CREATE TABLE streets (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  district_id INTEGER REFERENCES districts(id),
  active BOOLEAN DEFAULT true
);
-- Таблица коэффициентов времени доставки
CREATE TABLE delivery_time_multipliers (
  id SERIAL PRIMARY KEY,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  multiplier NUMERIC(3,2) NOT NULL DEFAULT
1.00,
  description VARCHAR(200),
  active BOOLEAN DEFAULT true

```



## ПРИЛОЖЕНИЕ Б

### Заполнение справочных данных

```
-- Категории блюд
INSERT INTO dish_categories (name, display_order,
active) VALUES
('Супы', 1, true),
('Салаты', 2, true),
('Горячие блюда', 3, true),
('Напитки', 4, true),
('Десерты', 5, true),
('Гарниры', 6, true),
('Блины', 7, true),
('Подарочные блюда', 8, true);
```

```
-- Статусы заказов
INSERT INTO order_statuses (id, name, description)
VALUES
```

```
(1, 'принят', 'Заказ принят в обработку'),
(2, 'готовится', 'Заказ готовится'),
(3, 'готов к выдаче', 'Заказ готов к выдаче'),
(4, 'выдан клиенту (самовывоз)', 'Заказ выдан
клиенту'),
(5, 'отменен', 'Заказ отменен'),
(6, 'готов к выдаче курьеру', 'Заказ готов к выдаче
курьеру'),
(7, 'забран курьером', 'Курьер забрал заказ'),
(8, 'на месте', 'Курьер на месте доставки'),
(9, 'вручен клиенту', 'Заказ вручен клиенту');
```

```
-- Роли сотрудников
INSERT INTO employee_roles (id, name, description,
permissions, active, base_salary, bonus) VALUES
(1, 'администратор', 'Полный доступ ко всем
функциям системы', '["all"]', true, 50000, 10000),
(2, 'сотрудник кухни', 'Доступ к кухонной панели и
управлению заказами',
'["kitchen_orders","menu_management"]', true, 35000,
5000),
(3, 'официант', 'Доступ к панели официанта,
бронированиям и заказам',
'["tables_management","orders_management"]', true,
30000, 5000),
(4, 'курьер', 'Доступ к приложению курьера',
'["delivery_orders"]', true, 25000, 3000);
```

```
-- Коэффициенты времени доставки
INSERT INTO delivery_time_multipliers (start_time,
end_time, multiplier, description, active) VALUES
('10:00:00', '15:59:59', 1.00, 'Дневное время', true),
```

```
('16:00:00', '23:00:00', 1.50, 'Вечернее время', true);
```

```
-- Города
INSERT INTO cities (name, base_delivery_price,
active) VALUES ('Казань', 300.00, true);
```

```
-- Районы
INSERT INTO districts (name, city_id,
delivery_surcharge, active) VALUES
('Вахитовский', 1, 0, true),
('Приволжский', 1, 300, true),
('Советский', 1, 300, true),
('Ново-Савиновский', 1, 300, true),
('Авиастроительный', 1, 400, true),
('Кировский', 1, 400, true);
```

```
-- Сотрудники (администратор и сотрудник кухни
для тестирования)
INSERT INTO employees (first_name, last_name,
position, phone, login, password, role_id, hire_date,
active) VALUES
('Иван', 'Петров', 'Администратор', '+79991112233',
'admin', 'admin123', 1, '2024-01-01', true),
('Мария', 'Сидорова', 'Шеф-повар', '+79994445566',
'cook', 'cook123', 2, '2024-01-01', true),
('Алексей', 'Кузнецов', 'Курьер', '+79997778899',
'courier1', '123456', 4, '2024-02-01', true);
```

```
-- Создание индексов для оптимизации
CREATE INDEX idx_orders_client_id ON
orders(client_id);
CREATE INDEX idx_orders_status_id ON
orders(status_id);
CREATE INDEX idx_orders_created_at ON
orders(created_at);
CREATE INDEX idx_order_items_order_id ON
order_items(order_id);
CREATE INDEX idx_user_levels_client_id ON
user_levels(client_id);
CREATE INDEX idx_user_xp_history_client_id ON
user_xp_history(client_id);
CREATE INDEX idx_xp_activities_client_active ON
xp_activities(client_id, is_active, expires_at);
```

## ПРИЛОЖЕНИЕ В

### Уровни лояльности и преимущества

| Уровень | Название | Требуемый опыт (XP) | Скидка на заказ |
|---------|----------|---------------------|-----------------|
| 1       | Новичок  | 0 – 199             | 5%              |
| 2       | Новичок  | 200 – 399           | 5%              |
| 3       | Новичок  | 400 – 599           | 5%              |
| 4       | Новичок  | 600 – 799           | 5%              |
| 5       | Новичок  | 800 – 999           | 5%              |
| 6       | Гурман   | 1000 – 1799         | 10%             |
| 7       | Гурман   | 1800 – 2599         | 10%             |
| 8       | Гурман   | 2600 – 3399         | 10%             |
| 9       | Гурман   | 3400 – 4199         | 10%             |
| 10      | Гурман   | 4200 – 4999         | 10%             |
| 11      | Эксперт  | 5000 – 6999         | 15%             |
| 12      | Эксперт  | 7000 – 8999         | 15%             |
| 13      | Эксперт  | 9000 – 10999        | 15%             |
| 14      | Эксперт  | 11000 – 12999       | 15%             |
| 15      | Эксперт  | 13000 – 14999       | 15%             |
| 16      | Легенда  | 15000 – 17999       | 20%             |
| 17      | Легенда  | 18000 – 20999       | 20%             |
| 18      | Легенда  | 21000 – 23999       | 20%             |
| 19      | Легенда  | 24000 – 26999       | 20%             |
| 20      | Легенда  | 27000 – 29999       | 20%             |
| 21      | Икона    | 30000+              | 25%             |